N048 GitHub Actions로 자동 배포하기_학습서

Chapter01

GitHub Actions로 자동 배포하기

이 교안에서도 지금까지 배운 요소를 FastAPI 하나로 묶은 AI 친구 에이전트 서비스를 예로 들어 실습합니다. 줄여서 에이전트 서비스라고 부르겠습니다. 에이전트 서비스는 이제 서비스 URL(배포하면 Cloud Run이 붙여 주는 공개 주소)에서 동작하지만, 고칠 때마다 누군가 로컬 환경(내 컴퓨터)에서 배포 명령을 실행해야 합니다. 명령을 실행한 사람만 무엇이 언제 배포되었는지 알고, 로컬 환경에 남아 있던 고치다 만 파일이 함께 올라가도 아무도 모릅니다.

Vercel에서는 GitHub 저장소에 코드를 올리면 배포가 저절로 시작되었습니다. 같은 일을 에이전트 서비스에도 걸어 보면 질문이 두 가지 생깁니다. GitHub에서 도는 프로그램이 내 Google Cloud 프로젝트에 배포할 권한은 어떻게 얻을까요. 그리고 머지만 하면 바로 사용자에게 나가는 구조라면, 잘못된 코드가 머지되었을 때는 어떤 일이 생길까요.

주관식 퀴즈

Q. [시작 질문] Vercel에 연결된 저장소에 푸시하면 몇 분 뒤 새 버전이 배포되었습니다. 그 몇 분 동안 Vercel은 무엇을 하고 있었을까요. 그리고 Vercel이 내 저장소의 코드를 가져갈 수 있었던 것은 어떤 권한 덕분이었을지 짐작해 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. CI와 CD가 각각 무엇을 자동으로 하는지 설명하고, 워크플로 파일에서 트리거(언제 실행할지), 러너(어디서 실행할지), 단계를 찾아 읽을 수 있습니다.
2. 간단한 워크플로를 직접 만들어 PR과 실행 버튼으로 돌리고, 실행 로그에서 단계별 결과와 실패한 단계를 찾을 수 있습니다.
3. 브랜치, 커밋, Pull Request(PR), 머지로 이어지는 변경 흐름을 GitHub 웹 화면에서 확인할 수 있습니다.
4. 키 파일 없이 GitHub Actions에 배포 권한을 주는 구조를 저장소, 서비스 계정, 역할로 나눠 설명할 수 있습니다.
5. 권한을 주는 명령을 실행하기 전에 키 파일, 기본 역할(basic role, Owner나 Editor처럼 프로젝트 전체에 넓은 권한을 주는 역할), 액션 버전, 삭제 여부를 점검할 수 있습니다.
6. 머지 한 번으로 배포된 결과를 워크플로 실행(workflow run) 로그, 리비전, `/version` 응답으로 이어서 확인할 수 있습니다.
7. 자동 배포가 막아 주는 문제와 그대로 사용자에게 내보내는 문제를 구분하고, 머지한 변경을 되돌릴 수 있습니다.

Chapter02

시작 상태 맞추기

이 교안은 에이전트 서비스가 Cloud Run에 배포되어 있고, LLM API 키(OpenAI, Google 같은 LLM 제공사가 발급하는 API 키)를 Secret Manager의 시크릿 `llm-api-key` 로 읽고 있는 상태에서 시작합니다. 시크릿(secret)은 API 키처럼 밖에 드러나면 안 되는 값을 말하고, Google Cloud 콘솔에서는 "보안 비밀"로 표시됩니다. 앞 실습에서 이 상태를 만들었다면 아래 확인 명령만 실행하고 넘어갑니다. 처음부터 시작한다면 시작 상태 문서의 A, B 항목을 먼저 따라 합니다. Google Cloud 가입과 gcloud 설치까지 포함하면 40분 안팎이 걸리므로 수업 전에 해 두기를 권합니다.

필요한 상태	확인 방법	없으면
gcloud 로그인, 프로젝트, 기본 리전 `us-central1`	`gcloud config list`	시작 상태 문서 A
템플릿으로 만든 내 공개 저장소를 내려받은 폴더	`git remote -v` 에 내 저장소 주소가 보임	시작 상태 문서 A
`friend-agent` 서비스가 시크릿 `llm-api-key` 를 참조해 배포됨	아래 명령에서 서비스 URL과 시크릿 이름이 나옴	시작 상태 문서 B

직접 실행할 명령: 시작 상태 확인

```
gcloud config list
git remote -v
gcloud run services describe friend-agent --region us-central1 --format "value(status.url)"
gcloud secrets describe llm-api-key --format "value(name)"
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 를 읽고, '자동 배포' 실습을 시작하는 데 필요한 상태가 갖춰져 있는지 확인만 해 줘.
아무것도 만들거나 바꾸지 말고, 갖춰진 항목과 빠진 항목을 표로 알려 줘. 빠진 항목은 문서의 어느 단계로 만들면 되는지도 적어 줘.
```

서비스 URL이 나오고, 그 URL을 브라우저로 열었을 때 캐릭터 만들기나 대화 화면이 보이면 준비가 끝난 것입니다.

지금 배포가 일어나는 방식

지금 에이전트 서비스를 고치면, 고친 사람이 자기 로컬 환경에서 배포 명령을 실행해야 사용자에게 반영됩니다. 혼자 작은 서비스를 운영할 때는 이 방식으로도 충분해 보이지만, 고치는 횟수가 늘고 고치는 사람이 둘 이상이 되면 곧 문제가 생깁니다. 무엇이 언제 배포되었는지는 명령을 실행한 사람의 터미널에만 남고, 로컬 환경에 있던 고치다 만 파일이나 커밋하지 않은 변경이 함께 올라가도 알아차리기 어렵습니다. 배포 전에 확인해야 할 것을 깜빡해도 막는 장치가 없고, 그 사람이 자리에 없으면 급한 수정도 배포할 수 없습니다.

아래 그림은 같은 변경이 사용자에게 나가기까지의 단계를 두 방식으로 나란히 놓은 것입니다. 위쪽이 지금의 수동 배포(사람이 직접 배포 명령을 실행하는 방식)이고, 아래쪽이 오늘 만들 CI/CD입니다.



그림: 수동 배포와 CI/CD 배포의 단계, 실행 주체, 남는 기록 비교

CI: 합치기 전에 매번 테스트하기

CI는 Continuous Integration, 우리말로 지속적 통합입니다. 여기서 통합은 여러 변경을 main 브랜치 하나로 합치는 일을 말합니다. 여러 사람이 각자 고친 코드를 한참 모았다가 한 번에 합치면, 무엇 때문에 깨졌는지 찾기 어려울 만큼 문제가 커져 있습니다. 그래서 변경을 작게 나눠 자주 합치고, 합치기 전마다 같은 테스트를 자동으로 돌려 문제를 작을 때 찾는 방식이 CI입니다.

CI가 돌리는 것은 보통 "이런 입력에는 이렇게 동작해야 한다"를 확인하는 테스트입니다. 사람이 기억해서 돌리는 테스트는 바쁠 때 빠지지만, PR이 올라올 때마다 GitHub이 같은 테스트를 돌리게 해 두면 빠질 일이 없습니다. 결과도 PR 화면에 남는데, PR 화면에 표시되는 CI 결과를 상태 체크(status check)라고 합니다.

CD: 합쳐진 코드를 매번 같은 방법으로 배포하기

CD는 두 가지 뜻으로 쓰입니다. 지속적 전달(Continuous Delivery)은 main 에 합쳐진 코드를 언제든 배포할 수 있는 상태로 만들어 두고, 실제 배포는 사람이 버튼을 눌러 결정하는 방식입니다. 지속적 배포(Continuous Deployment)는 사람의 결정 없이 main 에 합쳐지는 즉시 사용자에게 나가는 방식입니다. 오늘은 지속적 배포를 만들고, 사람의 승인을 거치게 하는 방법은 환경을 나누는 실습에서 다룹니다.

어느 쪽이든 핵심은 배포 방법이 사람의 손이 아니라 저장소의 파일에 적혀 있다는 점입니다. 누가 머지하든 같은 순서로 배포되고, 저장소에 커밋된 코드만 배포되며, 언제 어떤 커밋이 나갔는지 기록이 남습니다. 문제가 생겼을 때 되돌릴 커밋을 찾을 수 있는 것도 이 기록 덕분입니다.

AI 서비스에서 더 필요한 이유

AI 서비스는 코드보다 작은 변경이 자주 일어납니다. 지시문 한 줄, 모델 이름, 도구 설명, 확신도 기준 같은 값이 바뀌면 답이 달라지는데, 이런 변경은 화면만 봐서는 무엇이 달라졌는지 알아차리기 어렵습니다. 그래서 "언제 어떤 변경이 나갔는가"를 남기는 일이 일반 웹 서비스보다 더 중요하고, 이상한 답이 나왔을 때 그 시점의 변경을 찾아 되돌릴 수 있어야 합니다.

대가도 있습니다. 머지하면 바로 나가는 구조에서는 잘못된 코드도 머지하는 순간 사용자에게 나갑니다. 오늘 섹션 6에서 실제로 겪어 보고, 이 문제를 막는 CI 테스트는 다음 실습에서 붙입니다.

객관식 퀴즈

Q. "main 에 합쳐진 코드는 자동으로 배포할 준비까지 해 두고, 실제로 사용자에게 내보내는 것은 담당자가 승인 버튼을 눌러 정한다." 이 방식을 가장 잘 부르는 이름은 무엇인가요?

- 1. 지속적 통합(CI)
- 2. 지속적 전달(Continuous Delivery)
- 3. 지속적 배포(Continuous Deployment)
- 4. 수동 배포**제출**

- •

  1. 지속적 통합(CI)

- •

  2. 지속적 전달(Continuous Delivery)

- •

  3. 지속적 배포(Continuous Deployment)

- •

  4. 수동 배포

GitHub Actions

Vercel은 CI와 CD를 Vercel 안에서 대신 처리했기 때문에 따로 설정할 것이 없었습니다. Cloud Run에는 그런 연결이 기본으로 들어 있지 않으므로, 배포를 대신 실행해 줄 곳이 필요합니다. 이 교안에서는 GitHub에 들어 있는 자동 실행 기능인 GitHub Actions를 씁니다.

GitHub Actions는 저장소에 어떤 일이 생기면 미리 정해 둔 명령을 GitHub의 컴퓨터에서 실행합니다. 어떤 일이란 main 에 커밋이 들어온 것, PR이 열린 것, 정해 둔 시각이 된 것, 누군가 실행 버튼을 누른 것 같은 일입니다. 실행할 명령과 순서는 저장소 안의 `.github/workflows/` 폴더에 파일로 적어 두고, 이 파일을 워크플로라고 부릅니다. 테스트와 배포가 가장 흔한 쓰임새이고, 이슈 정리나 매일 정해진 시각에 도는 작업에도 씁니다.

실무에서 CI/CD 도구는 보통 코드를 어디에 두느냐로 정해집니다. 코드가 GitHub에 있는 팀은 GitHub Actions를 가장 흔히 고르고, GitLab에 두는 회사는 GitLab CI/CD를, 오래 운영해 온 큰 회사는 직접 설치해 쓰는 Jenkins를 쓰는 경우가 많습니다. Google Cloud Build나 AWS CodePipeline처럼 클라우드 회사가 만든 도구도 있습니다. 이 교안이 GitHub Actions를 쓰는 이유는 실습 저장소가 이미 GitHub에 있어 계정을 더 만들 필요가 없고, 공개 저장소에서는 무료이며, Google이 Cloud Run 배포용 공식 액션을 제공하기 때문입니다. 도구마다 파일 문법은 다르지만 언제 실행할지, 어디서 실행할지, 어떤 단계를 거칠지를 적는 구조는 같아서, 다른 도구를 만나도 오늘 배운 구조로 읽을 수 있습니다.

참고로 Cloud Run 콘솔에도 GitHub 저장소를 연결하면 Cloud Build가 자동으로 배포해 주는 메뉴가 있습니다. 연결은 더 간단하지만 배포 순서가 Google Cloud 화면 안에 들어 있어서, CI 테스트를 통과해야 배포하거나 승인을 거치게 하는 흐름을 저장소에서 직접 짜기는 어렵습니다.

워크플로를 이루는 말

워크플로 파일과 실행 화면에는 아래 말이 계속 나옵니다. 처음 보면 낯설지만, 각각 가리키는 대상이 하나씩입니다.

말	가리키는 것	워크플로 파일에서
워크플로	실행할 일 전체를 적은 파일 하나	`.github/workflows/deploy.yml`
트리거	워크플로를 시작하게 하는 이벤트	`on:` 아래 (예: main 에 push)
작업(job)	같은 컴퓨터에서 이어서 실행되는 단계 묶음	`jobs:` 아래 이름 (예: `deploy`)
단계(step)	작업 안에서 위에서 아래로 하나씩 실행되는 일	`steps:` 아래 항목
러너	작업을 실행하는 컴퓨터. 실행마다 새로 준비되고 끝나면 지워진다	`runs-on: ubuntu-latest`
액션	다른 사람이 만들어 공개한 단계 묶음. 가져다 쓴다	`uses: google-github-actions/deploy-cloudrun@v3`
변수와 시크릿	워크플로가 읽는 값. 저장소 설정에 넣는다	`${{ vars.GCP_PROJECT_ID }}`
워크플로 실행 (workflow run)	워크플로가 한 번 돈 기록과 단계별 로그	저장소의 Actions 탭

액션 이름 뒤의 `@v3` 는 버전입니다. 같은 액션이라도 버전에 따라 받는 값과 동작이 달라질 수 있어서, 템플릿에 적힌 버전을 바꾸지 않는 것을 작업 규칙으로 두었습니다. 액션이 무엇을 받는지는 그 액션의 공개 페이지에서 확인합니다.

직접 확인할 곳: Actions 탭과 액션 페이지 둘러보기

1. 내 저장소의 GitHub 화면 위쪽에서 Actions 탭을 엽니다. 아직 워크플로가 없으므로 시작용 워크플로를 고르라는 화면이 보입니다. 템플릿의 워크플로는 실행되지 않도록 `starter/workflows/` 에 따로 넣어 두었기 때문입니다.
2. https://github.com/google-github-actions/deploy-cloudrun 을 열어 이 액션이 받는 값(Inputs)의 목록에서 `service`, `region`, `source` 를 찾아봅니다. 오늘 워크플로에 이 세 값이 들어갑니다.

템플릿의 배포 워크플로 읽기

이제 오늘 쓸 워크플로를 위 말들로 나눠 읽어 봅니다. 오늘은 CD를 먼저 만들고, CI 테스트는 배포를 겪어 본 뒤에 붙입니다.



그림: 템플릿의 배포 워크플로를 트리거, 러너, 단계, 권한으로 나눠 읽기

그림 왼쪽은 템플릿의 `starter/workflows/deploy.yml` 을 줄인 것입니다. `on` 은 트리거로, main 브랜치에 커밋이 들어오면 시작합니다. `runs-on` 은 러너를 정하는 줄로, GitHub이 실행할 때마다 새로 준비하는 리눅스 컴퓨터에서 돌고 끝나면 그 컴퓨터는 지워집니다. 내 컴퓨터가 꺼져 있어도 배포가 진행되고, 로컬 환경에만 있던 파일은 배포에 섞이지 않습니다. `steps` 는 위에서 아래로 실행되는 단계이고, 한 단계라도 실패하면 거기서 멈춥니다.

`permissions` 의 `id-token: write` 는 오늘의 핵심입니다. 러너는 Google Cloud 계정이 없으므로, GitHub이 "이 토큰은 어느 저장소의 어느 워크플로에서 나왔다"고 서명한 OIDC 토큰(OpenID Connect 표준을 따르는 신원 토큰)을 받아 Google Cloud에 내밀어야 합니다. 이 토큰을 받는 권한이 `id-token: write` 이고, Google Cloud가 토큰을 받아 주도록 설정하는 일은 섹션 4에서 합니다. 공개 저장소에서는 GitHub이 제공하는 기본 러너를 무료로 쓸 수 있습니다(2026년 10월 기준).

객관식 퀴즈

Q. 템플릿의 배포 워크플로를 저장소에 넣은 뒤, 작업 브랜치에서 PR을 열기만 하고 아직 머지하지 않았습니다. 이때 배포가 일어날까요?

- 1. 일어난다. 저장소에 변경이 올라왔기 때문이다
- 2. 일어나지 않는다. 이 워크플로는 main 브랜치에 커밋이 들어올 때만 실행된다
- 3. 일어난다. PR을 열면 GitHub이 러너를 준비하기 때문이다
- 4. 일어나지 않는다. 워크플로는 사람이 Actions 탭에서 직접 실행해야만 돈다

- •

  1. 일어난다. 저장소에 변경이 올라왔기 때문이다

- •

  2. 일어나지 않는다. 이 워크플로는 main 브랜치에 커밋이 들어올 때만 실행된다

- •

  3. 일어난다. PR을 열면 GitHub이 러너를 준비하기 때문이다

-
-

4. 일어나지 않는다. 워크플로는 사람이 Actions 탭에서 직접 실행해야만 돈다

Chapter03

main에 바로 고치지 않는 이유

섹션 2의 워크플로를 넣고 나면 main 에 들어간 커밋은 곧바로 사용자에게 나갑니다. 그래서 main 에서 직접 고치지 않고, main 과 분리된 작업 공간인 브랜치에서 고친 뒤 "이 변경을 main 에 합쳐 달라"는 요청인 PR을 열고, 바뀐 줄을 확인한 다음 합칩니다(머지). PR 화면에는 어떤 파일의 어느 줄이 바뀌었는지가 남아서, 나중에 무엇이 언제 배포되었는지 찾을 때도 이 화면을 엽니다.

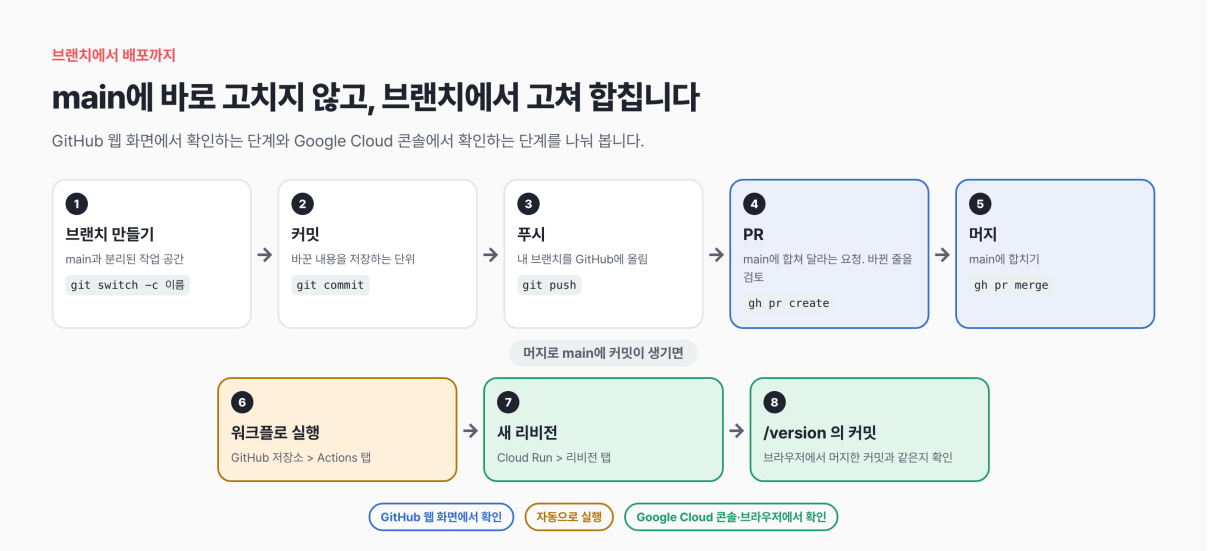


그림: 브랜치 만들기부터 머지까지, 그리고 머지 뒤에 자동으로 일어나는 일

그림의 1~5번을 배포와 상관없는 작은 변경으로 먼저 연습합니다. 아직 배포 워크플로가 없으므로 머지해도 6번 이후는 일어나지 않습니다. `README.md` 맨 아래에 이 저장소를 누가 어떤 실습용으로 만들었는지 한 줄을 추가하는 변경을 PR로 올려 봅니다. 브랜치 이름은 무엇을 하는 변경인지 알 수 있게 짓습니다.

직접 실행할 명령: 브랜치에서 고치고 PR 열기

```
# 최신 main에서 시작한다
git switch main
git pull

# 1. 브랜치 만들기
git switch -c readme-owner-note

# README.md 맨 아래에 한 줄을 추가한 뒤
# 2. 커밋
git add README.md
git commit -m "README에 저장소 설명 한 줄 추가"

# 3. 푸시: 내 브랜치를 GitHub에 올린다
git push -u origin readme-owner-note

# 4. PR 열기: main 에 합쳐 달라는 요청
```

```
gh pr create --base main --title "README에 저장소 설명 한 줄 추가" --body "README.md 맨 아래에 저장소 설명을 한 줄 추가한다"
```

코딩 에이전트에게 맡길 때: 브랜치에서 고치고 PR 열기

```
main을 최신으로 받은 뒤 readme-owner-note 브랜치를 만들어 줘.
README.md 맨 아래에 "이 저장소는 <내 이름>이 Cloud Run 배포 실습용으로 만들었다." 한 줄을 추가해 줘.
커밋하고 GitHub에 푸시한 다음, main 으로 가는 PR을 만들어 줘. 머지는 하지 마.
명령을 실행하기 전에 그 명령이 무엇을 하는지 한 줄씩 설명해 줘.
```

푸시와 PR 생성은 내 GitHub 저장소를 바꾸는 명령이므로, 에이전트에게 맡겼다면 실행 전에 브랜치 이름이 `readme-owner-note` 인지, 대상이 main 이 아니라 작업 브랜치인지 확인합니다. main 으로 바로 푸시하거나 `--force` 가 붙은 명령은 실행하지 않습니다.

PR 화면에서 확인하고 머지하기

PR이 만들어지면 브라우저에서 엽니다. 대화(Conversation) 탭에는 제목과 설명, 바뀐 파일(Files changed) 탭에는 바뀐 줄이 초록색(추가)과 빨간색(삭제)으로 보입니다. 바뀐 줄이 의도한 것뿐인지 확인했으면 머지합니다. 머지는 웹 화면의 Merge pull request 버튼으로 해도 되고, 아래 명령으로 해도 결과는 같습니다.

직접 실행할 명령: PR 열어 보고 머지하기

```
# PR을 브라우저로 연다
gh pr view --web

# 바뀐 줄을 확인했으면 머지하고, 다 쓴 작업 브랜치는 지운다
gh pr merge --merge --delete-branch

# 로컬 환경의 main 도 최신으로
git switch main
git pull
```

코딩 에이전트에게 맡길 때: 머지하기

```
방금 만든 PR을 브라우저로 열어 줘. 내가 바뀐 줄을 확인하고 말하면, 그때 머지하고 작업 브랜치를 지운 다음 로컬 환경의 main 을 최신으로 맞춰 줘.
```

머지가 끝나면 PR 상태가 Merged 로 바뀌고, 저장소 첫 화면의 커밋 기록 맨 위에 머지 커밋이 생깁니다. 오늘 이후로 배포되는 모든 변경은 이 흐름을 지나가므로, 무엇이 언제 바뀌었는지는 PR 목록의 Closed 탭에서 찾을 수 있습니다.

객관식 퀴즈

Q. 작업 브랜치에서 `git push` 까지 끝냈지만 PR은 아직 열지 않았습니다. 이 변경은 지금 어디에 있을까요?

- 1. 내 로컬 환경에만 있다
- 2. GitHub의 작업 브랜치에 올라가 있지만 main 에는 아직 합쳐지지 않았다

- 3. GitHub의 main 브랜치에 합쳐져 있다
- 4. PR을 열지 않았으므로 GitHub에 아무것도 올라가지 않았다**제출**

- •

  1. 내 로컬 환경에만 있다

- •

  2. GitHub의 작업 브랜치에 올라가 있지만 main 에는 아직 합쳐지지 않았다

- •

  3. GitHub의 main 브랜치에 합쳐져 있다

- •

  4. PR을 열지 않았으므로 GitHub에 아무것도 올라가지 않았다

첫 워크플로 실행하기

브랜치와 PR 흐름을 익혔으니, 섹션 2에서 읽기만 한 워크플로를 직접 하나 만들어 돌려 봅니다. 배포 워크플로는 Google Cloud 권한이 있어야 돌기 때문에, 먼저 Google Cloud에 아무것도 하지 않는 연습용 워크플로 `hello.yml` 을 씁니다. 이 워크플로는 러너의 운영체제와 Python 버전을 출력하고, 저장소 코드를 내려받은 뒤 받은 파일 목록을 보여 줍니다. 트리거는 두 가지입니다. `pull_request` 는 PR이 열리거나 PR에 커밋이 더해질 때, `workflow_dispatch` 는 Actions 탭에서 사람이 실행 버튼을 누를 때 워크플로를 시작합니다.

파일 내용은 아래와 같습니다. 러너는 실행마다 새로 준비되므로, 첫 단계에서 코드를 내려받기 전에 `ls` 를 한 번 실행해 러너가 비어 있는지 보고, 내려받은 뒤에 다시 `ls` 를 실행해 무엇이 왔는지 봅니다. 마지막 단계는 로컬 환경에만 있는 `.env` 파일이 러너에 있는지 확인합니다.

`.github/workflows/hello.yml`

```
name: hello

on:
pull_request:
workflow_dispatch:

permissions:
contents: read

jobs:
hello:
runs-on: ubuntu-latest
steps:
-name: 러너 정보 보기
run:|
          echo "시작한 이벤트: ${{ github.event_name }}"
          echo "브랜치: ${{ github.head_ref || github.ref_name }}"
          uname -a
          python3 --version
          echo "코드를 내려받기 전 파일 목록:"
          ls -a

-name: 저장소 코드 내려받기
uses: actions/checkout@v7

-name: 내려받은 파일 보기
run:|
```

```
          ls
          if [ -f .env ]; then echo ".env 있음"; else echo ".env 없음"; fi
```

직접 실행할 명령: 연습용 워크플로를 PR로 올리기

```
git switch main
git pull
git switch -c add-hello-workflow

# .github/workflows/hello.yml 을 위 내용으로 만든 뒤
git add .github/workflows/hello.yml
git commit -m "연습용 hello 워크플로 추가"
git push -u origin add-hello-workflow
gh pr create --base main --title "연습용 hello 워크플로 추가" --body "러너 정보와 내려받은 파일을 출력하는 워크플
로"

# PR에서 시작된 실행을 끝날 때까지 지켜본다
gh run watch
```

코딩 에이전트에게 맡길 때: 연습용 워크플로를 PR로 올리기

```
main을 최신으로 받은 뒤 add-hello-workflow 브랜치를 만들고, 교안에 있는 내용 그대로 .github/workflows/hello.yml
을 만들어 줘.
액션 버전(actions/checkout@v7)과 트리거, permissions 는 바꾸지 마. 시크릿이나 Google Cloud 관련 단계는 넣지 마.
커밋하고 푸시한 다음 main 으로 가는 PR을 만들어 줘. 머지는 하지 마.
```

에이전트가 만든 파일을 커밋하기 전에 세 가지를 봅니다. 트리거가 `pull_request` 와 `workflow_dispatch` 두 개뿐인지, `permissions` 가 `contents: read` 하나인지, `uses:` 줄의 액션 버전이 템플릿의 배포 워크플로와 같은 `@v7` 인지 확인합니다.

실행 결과 확인하기

PR을 만들면 몇 초 뒤 PR 화면 아래쪽에 `hello` 상태 체크가 노란 점(실행 중)으로 나타나고, 1분 안팎이면 초록 체크로 바뀝니다. 워크플로 파일이 PR에 들어 있기만 해도 그 PR에서 바로 실행되는 것은 트리거가 `pull_request` 이기 때문입니다. 상태 체크 옆의 Details를 누르거나 Actions 탭으로 가면 같은 실행의 로그가 열립니다.

직접 확인할 곳: hello 실행 로그 읽기

1. PR 화면 아래쪽 상태 체크 목록에서 `hello` 옆의 Details를 누릅니다. Actions 탭 > 왼쪽 목록의 `hello` > 맨 위 실행으로 가도 같은 화면입니다.
2. 왼쪽의 작업 이름 `hello` 를 누르면 단계가 위에서 아래로 나열됩니다. 맨 앞의 Set up job 과 맨 끝의 Complete job 은 GitHub이 러너를 준비하고 치우는 단계입니다.
3. `러너 정보 보기` 를 펼쳐 이벤트 이름이 `pull_request` 인지, 운영체제가 Ubuntu인지, Python 버전이 무엇인지 보고, 코드를 내려받기 전 파일 목록에 `.` 과 `..` 말고는 아무것도 없는지 봅니다.
4. `내려받은 파일 보기` 를 펼쳐 저장소의 `app` , `static` , `starter` 폴더가 보이는지, 마지막 줄이 `.env 없음` 인지 봅니다.

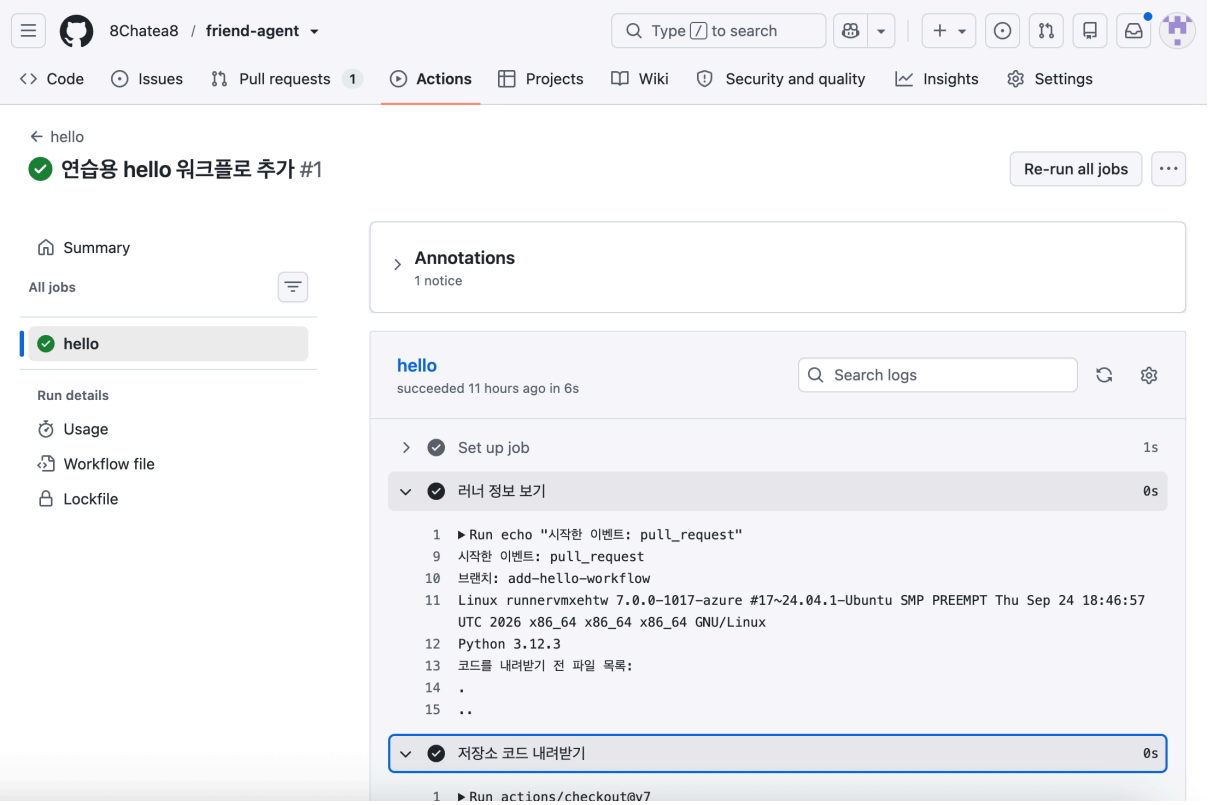


그림: GitHub 저장소 > Actions > hello > 맨 위 실행 > hello 작업: 단계 목록과 함께 '러너 정보 보기', '내려받은 파일 보기' 단계를 펼친 로그

코드를 내려받기 전 목록에 현재 폴더(`.`)와 상위 폴더(`..`) 표시만 있는 것은 러너가 실행마다 새로 준비되는 컴퓨터라서입니다. `actions/checkout` 단계가 GitHub에 올라간 커밋을 내려받은 뒤에야 저장소 파일이 생기고, 그 목록에 `.env` 는 없습니다. `.env` 는 `.gitignore` 에 적혀 있어 커밋되지 않았으므로 러너로 오지 않습니다. 배포 워크플로도 같은 러너에서 같은 방식으로 코드를 받으므로, 로컬 환경에만 있는 파일이나 커밋하지 않은 변경은 배포에 섞이지 않습니다. Python 버전이 로컬 환경과 다르게 나왔다면, 러너의 기본 Python은 GitHub이 러너 이미지에 정해 둔 값이고 저장소의 `.python-version` 을 따르지 않기 때문입니다. 러너에서 특정 버전이 필요하면 그 버전을 설치하는 단계를 워크플로에 따로 적어야 합니다.

실패한 단계 보기

섹션 2에서 단계는 위에서 아래로 실행되고 하나라도 실패하면 거기서 멈춘다고 읽었습니다. 같은 PR 브랜치에서 `러너 정보 보기` 단계의 `run:` 맨 아래에 `exit 1` 한 줄을 넣어 커밋하고 푸시합니다. `exit 1` 은 이 명령이 실패했다고 알리는 줄입니다. PR에 커밋이 더해졌으므로 `hello` 가 다시 실행되고, 이번에는 빨간 X로 끝납니다.

직접 실행할 명령: 단계 하나를 실패시키고 되돌리기

```
# hello.yml 의 '러너 정보 보기' 단계 맨 아래에 exit 1 을 넣은 뒤
git commit -am "hello 워크플로 실패 확인"
git push
gh run watch

# 확인이 끝나면 방금 커밋을 되돌린다
git revert --no-edit HEAD
```

```
git push
gh run watch
```

코딩 에이전트에게 맡길 때: 단계 하나를 실패시키고 되돌리기

```
add-hello-workflow 브랜치의 .github/workflows/hello.yml 에서 '러너 정보 보기' 단계의 run 맨 아래에 exit 1 한 줄만 추가해서 커밋하고 푸시해 줘.
실행이 끝나면 어느 단계가 실패했고 그 뒤 단계는 어떻게 됐는지 로그를 근거로 알려 줘. 내가 확인하면 그 커밋을 git revert 로 되돌려 푸시해 줘.
```

실패한 실행을 열면 `러너 정보 보기` 단계에 빨간 표시와 `Process completed with exit code 1` 줄이 있고, 그 아래 `저장소 코드 내려받기` 와 `내려받은 파일 보기` 는 회색으로 건너뛰어졌습니다. PR 화면의 상태 체크도 빨간 X로 바뀌었지만 Merge pull request 버튼은 그대로 눌립니다. 상태 체크는 결과를 보여 줄 뿐 머지를 막지 않기 때문입니다. 실패한 PR의 머지를 막는 설정은 CI 테스트를 붙이는 실습에서 다룹니다. 되돌린 커밋을 푸시하면 다시 초록 체크가 나옵니다.

직접 실행 버튼으로 돌리기

초록 체크를 확인했으면 PR을 머지합니다. `workflow_dispatch` 트리거는 워크플로 파일이 main 에 있어야 Actions 탭에 실행 버튼이 생기므로, 머지한 뒤에 한 번 직접 실행해 봅니다. 아직 배포 워크플로가 없으므로 이 머지로는 배포가 일어나지 않습니다.

직접 실행할 명령: 머지하고 직접 실행하기

```
gh pr merge --merge --delete-branch
git switch main
git pull

# Actions 탭의 Run workflow 버튼과 같은 일을 한다
gh workflow run hello.yml
gh run watch
```

직접 확인할 곳: Actions 탭에서 직접 실행하기

1. Actions 탭 > 왼쪽 목록의 `hello` 를 누르면 오른쪽 위에 Run workflow 버튼이 있습니다. 버튼을 누르고 브랜치를 `main` 으로 둔 채 실행합니다.
2. 새 실행을 열어 `러너 정보 보기` 의 이벤트 이름이 이번에는 `workflow_dispatch` 로 나오는지 봅니다.
3. 실행 목록에서 PR로 시작된 실행과 직접 시작한 실행이 트리거 이름과 함께 나란히 남아 있는지 봅니다.

`hello.yml` 은 PR마다 1분 안팎 돌고, 공개 저장소에서는 기본 러너가 무료라서 그대로 두어도 됩니다. 섹션 5에서 넣을 배포 워크플로는 `hello.yml` 과 같은 구조에 Google Cloud 인증과 배포 단계가 더해진 파일이고, 트리거만 main 에 대한 push 로 다릅니다.

객관식 퀴즈

Q. `hello` 실행 로그의 마지막 줄에 `.env 없음` 이 나왔습니다. 로컬 환경에는 분명히 `.env` 파일이 있는데 러너에 없는 이유로 가장 알맞은 것은 무엇인가요?

- 1. GitHub이 보안 때문에 이름이 `.env` 인 파일을 자동으로 지우기 때문이다
- 2. 러너는 GitHub에 올라간 커밋만 내려받고, `.env` 는 `.gitignore` 에 적혀 있어 커밋되지 않았기 때문이

다
• 3. `ls` 명령은 점으로 시작하는 파일을 보여 주지 않기 때문이다
• 4. `permissions` 가 `contents: read` 라서 숨김 파일을 읽을 수 없기 때문이다

- 1. GitHub이 보안 때문에 이름이 `.env` 인 파일을 자동으로 지우기 때문이다
- 2. 러너는 GitHub에 올라간 커밋만 내려받고, `.env` 는 `.gitignore` 에 적혀 있어 커밋되지 않았기 때문이다
- 3. `ls` 명령은 점으로 시작하는 파일을 보여 주지 않기 때문이다
- 4. `permissions` 가 `contents: read` 라서 숨김 파일을 읽을 수 없기 때문이다

Chapter04

배포 권한 구조 한눈에 보기

GitHub Actions 러너는 Google Cloud 계정이 없으므로, 러너가 배포하려면 Google Cloud가 "이 요청은 배포해도 되는 쪽에서 왔다"고 믿을 근거가 있어야 합니다. 이 섹션에서 만드는 설정을 한 문장으로 줄이면, 내 Google Cloud 프로젝트에 "GitHub Actions가 발급한 토큰 가운데 내 레포에서 온 것만 받는다"는 공급자를 만들고, 그렇게 들어온 신원에게 배포 서비스 계정을 쓸 권한을 주는 것입니다. 그러면 내 레포의 워크플로만, 배포할 때마다 새로 받는 짧은 토큰으로 배포 서비스 계정의 권한을 빌려 배포할 수 있습니다. 서비스 계정 키 파일처럼 오래 남는 값은 GitHub에도 Google Cloud에도 저장하지 않습니다. 아래 그림은 이 설정을 마친 뒤 배포할 때마다 일어나는 일을 차례로 그린 것입니다. 그림에 나오는 공급자, 서비스 계정 같은 말은 아래에서 하나씩 만들기 직전에 설명합니다.

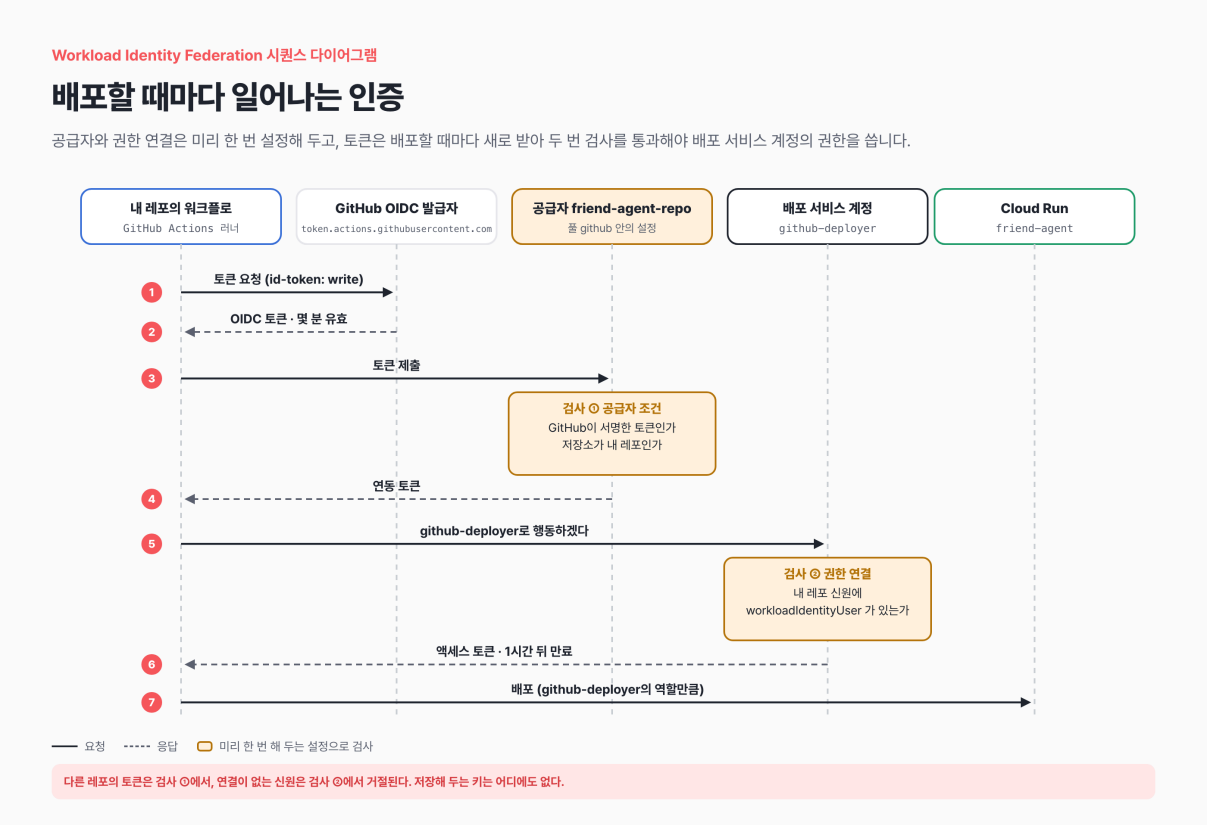


그림: 배포할 때마다 러너가 OIDC 토큰을 받아 공급자 조건과 권한 연결 두 번의 검사를 통과한 뒤 배포 서비스 계정의 권한으로 배포하는 시퀀스 다이어그램

그림의 번호를 따라가면 배포 한 번에 일어나는 일이 보입니다. 러너는 GitHub에 OIDC 토큰을 요청해 받고(1, 2), 그 토큰을 공급자 `friend-agent-repo` 에 제출합니다(3). 공급자는 GitHub이 서명한 토큰인지와 토큰의 저장소가 내 레포인지 확인하고, 통과하면 Google Cloud 안에서 쓰는 연동 토큰(federated token)을 돌려줍니다(4). 러너가 이 연동 토큰으로 배포 서비스 계정 `github-deployer` 로 행동하겠다고 요청하면(5), Google Cloud는 `github-deployer` 에 "내 레포에서 온 신원은 이 계정으로 행동할 수 있다"는 권한 연결이 있는지 확인하고 1시간 동안 쓸 수 있는 액세스 토큰을 줍니다(6). 마지막 배포(7)는 이 액세스 토큰, 곧 `github-deployer` 에 준 역할만큼의 권한으로 진행됩니다.

검사가 두 번인 이유는 확인하는 대상이 다르기 때문입니다. 공급자 조건은 토큰이 Google Cloud 안으로 들어올 수 있는지를 정하고, 권한 연결은 들어온 신원이 어느 서비스 계정을 쓸 수 있는지를 정합니다. 그래서 뒤의 설정 명령에는 내 레포 이름이 공급자 조건과 권한 연결 두 곳에 들어갑니다.

이 섹션에서 설정하는 것

그림의 흐름이 돌아가려면 Google Cloud와 GitHub에 아래 여섯 가지를 차례로 만들어 두어야 합니다. 앞의 것이 있어야 뒤의 것을 만들 수 있으므로 순서대로 진행하고, 명령은 모두 템플릿 저장소의 `docs/wif-setup.md` 에 단계별로 적혀 있습니다.

순서	만드는 것	어디에	그림에서 맡는 일	문서 단계
1	런타임 서비스 계정 `friend-agent-runtime` 과 시크릿 읽기 권한, 빌드 서비스 계정	Google Cloud IAM	서비스가 실행될 때와 소스를 빌드할 때 쓰는 계정 (그림 밖)	1

순서	만드는 것	어디에	그림에서 맡는 일	문서 단계
	`friend-agent-builder` 와 빌드 역할			
2	배포 서비스 계정 `github-deployer` 와 권한 네 개	Google Cloud IAM	7번 배포에 쓰이는 권한	2
3	풀 `github`	Google Cloud IAM	공급자를 담는 묶음	3
4	공급자 `friend-agent-repo` 와 조건	풀 안	3, 4번과 검사 ①	3
5	권한 연결 (내 레포에서 온 신원이 배포 서비스 계정을 쓸 권한)	`github-deployer` 계정	5, 6번과 검사 ②	4
6	저장소 변수 다섯 개	GitHub 저장소 설정	워크플로가 3번과 5번에서 공급자와 계정 이름을 읽는 곳	7

아래 그림은 1~5번에서 만드는 것을 계정 단위로 묶어 그린 것입니다. 위쪽 줄이 배포할 때 쓰이는 것(공급자와 배포 서비스 계정)이고, 아래쪽 줄이 배포된 서비스가 실행될 때 쓰이는 것(런타임 서비스 계정)입니다. 설정을 마친 뒤 콘솔에서 확인할 때도 이 그림의 이름과 역할을 그대로 대조합니다.



그림: 내 저장소의 워크플로가 OIDC 토큰으로 배포 서비스 계정을 가장하고, 서비스는 런타임 서비스 계정으로 실행된다

실행 전 점검 목록

`docs/wif-setup.md` 의 명령은 계정을 만들고 권한을 주는 것이라, 직접 치든 코딩 에이전트가 실행하든 명령마다 아래 표로 먼저 확인합니다. 에이전트는 오류가 나면 더 넓은 역할을 주는 쪽으로 해결하려 할 때가 많아서, 권한 오류가 났을 때 특히 이 표가 필요합니다.

확인할 것	걸리면
서비스 계정 키 파일(JSON)을 만드는가 (`keys create`)	실행하지 않는다. 키 파일은 만료 기한이 없어 새어 나가면 계속 쓰일 수 있으므로, WIF로 하라고 다시 요청한다

확인할 것	걸리면
Owner, Editor 같은 기본 역할을 주는가	실행하지 않는다. `docs/wif-setup.md` 의 역할 표와 대조한다
권한을 주는 대상이 프로젝트 전체인가, 하나인가	`serviceAccountUser` 는 런타임 서비스 계정과 빌드 서비스 계정 두 개에만, 시크릿 읽기는 `llm-api-key` 하나에만 준다
템플릿과 다른 버전의 액션을 쓰는가 (`@v1`, `@v2` 등)	실행하지 않는다. 템플릿의 버전을 유지한다
무언가를 삭제하는가	지우는 대상의 이름을 확인하고 판단한다

에이전트에게 주는 권한 설정(Claude Code에서는 `.claude/settings.json`)에는 키 파일 생성이 금지 명령으로, 권한 부여와 서비스 계정 생성이 매번 확인할 명령으로 들어 있습니다. 명령을 직접 친다면 같은 표가 그대로 점검 목록입니다.

서비스 계정 세 개 만들기

설정하는 것 1, 2번입니다. 서비스 계정(service account)은 사람이 아니라 프로그램이 쓰는 Google Cloud 계정입니다. `github-deployer@<프로젝트 ID>.iam.gserviceaccount.com` 처럼 이메일 주소로 구분하고, 비밀번호로 로그인하지 않으며, 할 수 있는 일은 계정에 준 역할로 정해집니다. 지금까지 `gcloud` 명령은 내 Google 계정의 권한으로 실행했지만, GitHub의 러너나 Cloud Run에서 도는 서비스는 내 계정으로 로그인할 수 없으므로 서비스 계정을 하나씩 정해 주고 그 권한으로 행동하게 합니다.

계정을 두 개로 나누는 이유는 쓰이는 때와 필요한 권한이 겹치지 않기 때문입니다. 런타임 서비스 계정(서비스가 실행되는 동안 쓰는 서비스 계정) `friend-agent-runtime` 은 요청을 처리하는 동안 내내 쓰이고, 시크릿 `llm-api-key` 하나를 읽는 권한만 있으면 됩니다. 배포 서비스 계정 `github-deployer` 는 머지할 때마다 몇 분 쓰이고, 배포에 필요한 역할만 있으면 됩니다. 하나로 합치면 인터넷에서 요청을 받는 서비스가 배포 권한까지 갖게 되고, 워크플로 파일에 한 줄만 추가해도 LLM API 키를 꺼낼 수 있게 됩니다. 필요한 일에 필요한 권한만 주는 이 원칙을 최소 권한 원칙(principle of least privilege)이라고 합니다. 지금 에이전트 서비스가 쓰는 기본 서비스 계정은 프로젝트의 다른 서비스도 함께 쓰는 계정이라, 그 계정에 걸린 키 읽기 권한도 섹션 5에서 회수합니다.

세 번째 계정은 빌드 서비스 계정입니다. 소스 배포를 하면 Cloud Build가 소스에서 이미지를 만드는데, 이 빌드를 실행하는 계정을 빌드 서비스 계정이라고 합니다. 따로 정하지 않으면 Compute Engine 기본 서비스 계정으로 빌드하는데, 조직 없이 만든 프로젝트에서는 이 계정에 프로젝트 대부분을 바꿀 수 있는 Editor 역할이 붙어 있는 경우가 많습니다. 배포 서비스 계정이 그 계정으로 빌드를 시작할 수 있으면 사실상 Editor 권한을 갖게 되므로, 빌드와 배포에 필요한 역할만 가진 빌드 서비스 계정 `friend-agent-builder` 를 따로 만들고 워크플로가 이 계정으로 빌드하게 합니다.

배포 서비스 계정에는 배포에 필요한 권한 네 개(소스 배포, Google Cloud API 사용, 새 리비전을 런타임 서비스 계정으로 실행하도록 지정하기, 빌드 서비스 계정으로 빌드 실행하기)를 줍니다. 그림의 5, 6번처럼 다른 서비스 계정의 권한으로 잠시 행동하는 것을 서비스 계정 가장(impersonation)이라고 합니다.

아래 명령은 문서의 0~2단계를 그대로 옮긴 것입니다. 맨 위 세 줄의 `<PROJECT_ID>` 와 저장소 자리만 내 값으로 바꾼 뒤 같은 터미널에서 위에서부터 차례로 실행하거나, 아래 요청 예시로 코딩 에이전트에게 맡깁니다. 각 명령의 옵션은 몰라도 되고, 명령마다 무엇을 만들거나 누구에게 권한을 주는지만 주석으로 확인합니다.

직접 실행할 명령: 서비스 계정 세 개 만들기

```
# 내 값으로 바꿀 세 줄
PROJECT_ID=<PROJECT_ID>
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

```
REPO=<GitHub 계정>/<저장소 이름>

# 런타임 서비스 계정과, 그 계정의 시크릿 하나 읽기 권한
gcloud iam service-accounts create friend-agent-runtime --project $PROJECT_ID --display-name "friend-age
nt 실행용"
gcloud secrets add-iam-policy-binding llm-api-key --project $PROJECT_ID --member "serviceAccount:friend-
agent-runtime@$PROJECT_ID.iam.gserviceaccount.com" --role roles/secretmanager.secretAccessor

# 빌드 서비스 계정과, 빌드한 이미지를 배포하는 역할
gcloud iam service-accounts create friend-agent-builder --project $PROJECT_ID --display-name "friend-age
nt 빌드용"
gcloud projects add-iam-policy-binding $PROJECT_ID --member "serviceAccount:friend-agent-builder@$PROJEC
T_ID.iam.gserviceaccount.com" --role roles/run.builder

# 배포 서비스 계정과 역할
gcloud iam service-accounts create github-deployer --project $PROJECT_ID --display-name "GitHub Actions
배포용"
DEPLOYER=github-deployer@$PROJECT_ID.iam.gserviceaccount.com
gcloud projects add-iam-policy-binding $PROJECT_ID --member "serviceAccount:$DEPLOYER" --role roles/run.
sourceDeveloper
gcloud projects add-iam-policy-binding $PROJECT_ID --member "serviceAccount:$DEPLOYER" --role roles/serv
iceusage.serviceUsageConsumer

# 배포 서비스 계정이 런타임 서비스 계정을 지정할 수 있게
gcloud iam service-accounts add-iam-policy-binding friend-agent-runtime@$PROJECT_ID.iam.gserviceaccount.
com --project $PROJECT_ID --member "serviceAccount:$DEPLOYER" --role roles/iam.serviceAccountUser

# 배포 서비스 계정이 빌드 서비스 계정으로 빌드를 실행할 수 있게
gcloud iam service-accounts add-iam-policy-binding friend-agent-builder@$PROJECT_ID.iam.gserviceaccount.
com --project $PROJECT_ID --member "serviceAccount:$DEPLOYER" --role roles/iam.serviceAccountUser
```

코딩 에이전트에게 맡길 때: 서비스 계정 세 개 만들기

```
docs/wif-setup.md 의 0~2단계를 순서대로 진행해 줘. 프로젝트 ID는 <PROJECT_ID>, 저장소는 <GitHub 계정>/<저장소 이
름> 이야.
명령 하나를 실행하기 전마다 무엇을 만들고 누구에게 어떤 역할을 주는지 설명하고 내 확인을 받아 줘.
문서에 없는 역할은 주지 말고, 서비스 계정 키 파일은 만들지 마. 시크릿 값은 읽거나 출력하지 마.
```

에이전트에게 맡겼다면 명령마다 앞의 실행 전 점검 목록으로 확인합니다. 특히 권한을 주는 대상이 프로젝트 전체가 아니라 계정 하나나 시크릿 하나인지 봅니다.

풀과 공급자 만들고 저장소 연결하기

설정하는 것 3~5번입니다. 이 세 가지를 묶어 Workload Identity Federation(WIF)이라고 부르며, Google Cloud 바깥의 프로그램이 키 파일 없이 서비스 계정의 권한을 쓰게 하는 기능입니다.

풀(Workload Identity Pool)은 Google Cloud 바깥에서 오는 신원을 받아들이는 묶음입니다. 풀 자체에는 권한이 없고, GitHub Actions에서 오는 신원을 담을 묶음 `github` 을 만들어 두는 단계입니다.

공급자(Workload Identity Pool Provider)는 풀 안에 두는 설정으로, 바깥에서 온 토큰을 받아들일지 정하는 규칙을 적습니다. 공급자에는 토큰을 미리 등록하지 않고, "GitHub Actions가 발급한 토큰이고 저장소가 내 레포일 때만 받는다"는 규칙만 적어 둡니다. 이 규칙이 그림의 검사 ①입니다.

마지막으로 권한 연결을 만듭니다. 풀로 들어온 신원 가운데 내 레포에서 온 것만 `github-deployer` 로 행동할 수 있게 `github-deployer` 계정에 권한을 하나 주는 것이고, 이 연결이 그림의 검사 ②입니다.

아래 명령은 문서의 3~4단계입니다. 앞에서 정한 변수를 그대로 쓰므로 같은 터미널에서 이어서 실행하거나 코딩 에이전트에게 맡깁니다.

직접 실행할 명령: 풀과 공급자, 저장소 연결

```
# 풀 만들기
gcloud iam workload-identity-pools create github --project $PROJECT_ID --location global --display-name
"GitHub Actions"

# 공급자 만들기: 내 저장소에서 온 토큰만 받는 규칙
gcloud iam workload-identity-pools providers create-oidc friend-agent-repo --project $PROJECT_ID --locat
ion global --workload-identity-pool github --display-name "friend-agent 저장소" --issuer-uri "https://tok
en.actions.githubusercontent.com" --attribute-mapping "google.subject=assertion.sub,attribute.repository
=assertion.repository,attribute.repository_owner=assertion.repository_owner" --attribute-condition "asse
rtion.repository=='$REPO'"

# 내 저장소의 워크플로가 배포 서비스 계정을 가장할 수 있게
gcloud iam service-accounts add-iam-policy-binding $DEPLOYER --project $PROJECT_ID --role roles/iam.work
loadIdentityUser --member "principalSet://iam.googleapis.com/projects/$PROJECT_NUMBER/locations/global/w
orkloadIdentityPools/github/attribute.repository/$REPO"
```

코딩 에이전트에게 맡길 때: 풀과 공급자, 저장소 연결

```
docs/wif-setup.md 의 3~4단계를 진행해 줘. 공급자의 속성 조건에는 내 저장소 이름 하나만 들어가야 해.
명령을 실행하기 전에 공급자 규칙과 권한 연결에 들어간 저장소 이름을 나에게 먼저 보여 줘.
```

실행 전에는 명령에 들어간 저장소 이름이 내 저장소(`<GitHub 계정>/<저장소 이름>`)와 같은지만 확인합니다. 계정 이름까지만 적혀 있으면 그 계정의 모든 저장소가 배포할 수 있게 됩니다.

저장소에 값 넣기와 확인

워크플로는 프로젝트 ID, 공급자 이름, 세 서비스 계정 주소를 저장소 변수에서 읽습니다. 다섯 값 모두 비밀이 아니므로 Secrets 가 아니라 Variables 에 넣습니다. 그다음 문서 8단계대로 콘솔에서 결과를 확인합니다.

직접 실행할 명령: 저장소 변수 넣기

```
WIF_PROVIDER=$(gcloud iam workload-identity-pools providers describe friend-agent-repo --project $PROJEC
T_ID --location global --workload-identity-pool github --format 'value(name)')

gh variable set GCP_PROJECT_ID --body "$PROJECT_ID"
gh variable set WIF_PROVIDER --body "$WIF_PROVIDER"
gh variable set DEPLOY_SA --body "$DEPLOYER"
gh variable set RUNTIME_SA --body "friend-agent-runtime@$PROJECT_ID.iam.gserviceaccount.com"
gh variable set BUILD_SA --body "friend-agent-builder@$PROJECT_ID.iam.gserviceaccount.com"
gh variable list
```

코딩 에이전트에게 맡길 때: 저장소 변수 넣기

```
docs/wif-setup.md 의 7단계대로 GitHub 저장소 변수 다섯 개(GCP_PROJECT_ID, WIF_PROVIDER, DEPLOY_SA, RUNTIME_S
A, BUILD_SA)를 넣고, 마지막에 변수 목록을 보여 줘.
```

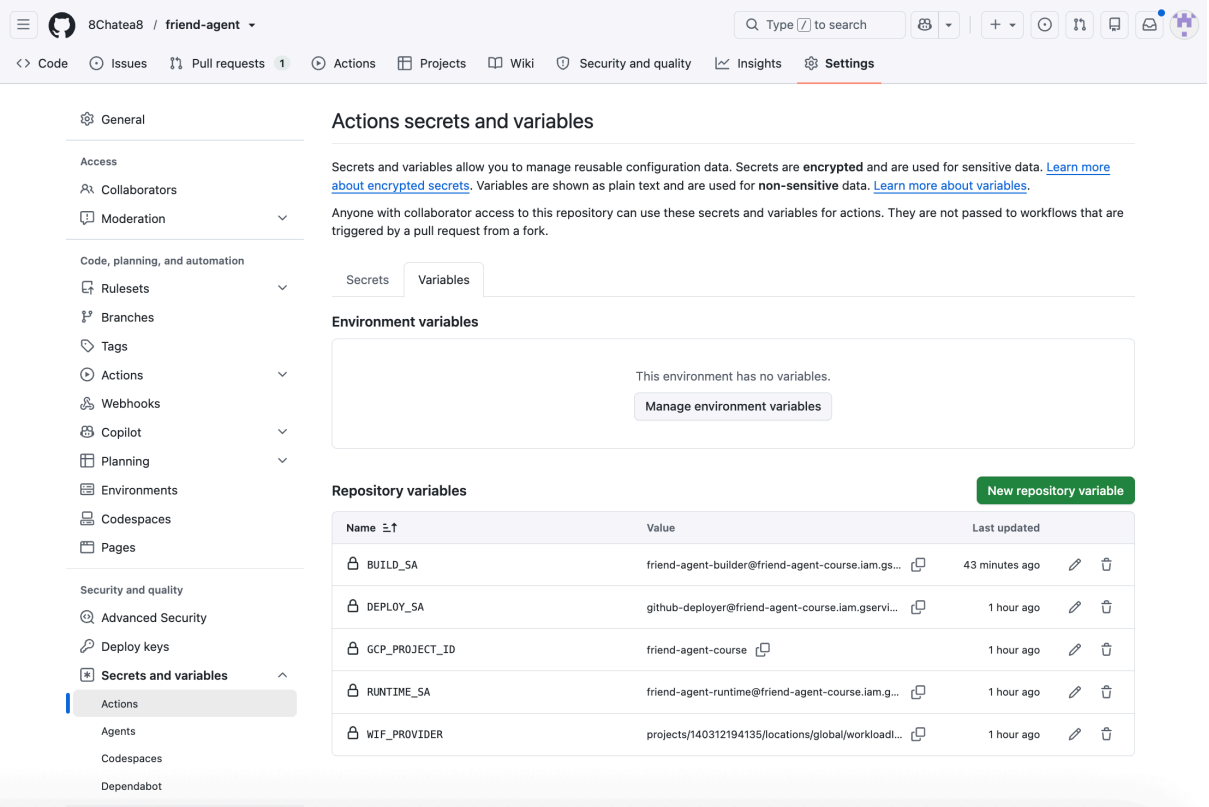


그림: GitHub 저장소 > Settings > Secrets and variables > Actions > Variables 탭: GCP_PROJECT_ID, WIF_PROVIDER, DEPLOY_SA, RUNTIME_SA, BUILD_SA 다섯 변수가 보이는 모습

직접 확인할 곳: Google Cloud 콘솔에서 확인

1. IAM 및 관리자 > 워크로드 아이덴티티 제휴(Workload Identity Federation)를 엽니다. 목록에는 ID 대신 표시 이름이 나오므로 풀 `GitHub Actions` 를 누르고, 공급자 목록에서 `friend-agent 저장소` 옆의 수정(연필) 버튼을 누릅니다. 공급자 수정 화면의 속성 조건(Attribute conditions)에 내 저장소 이름이 있는지 보고, 아무것도 바꾸지 않고 취소합니다.
2. IAM 및 관리자 > 서비스 계정에서 `github-deployer` , `friend-agent-runtime` , `friend-agent-builder` 를 열고, 키(Keys) 탭에 키가 하나도 없는지 봅니다.
3. IAM 및 관리자 > IAM에서 `github-deployer` 의 역할이 소스 배포와 API 사용 두 개뿐이고 Owner, Editor 가 없는지 봅니다.

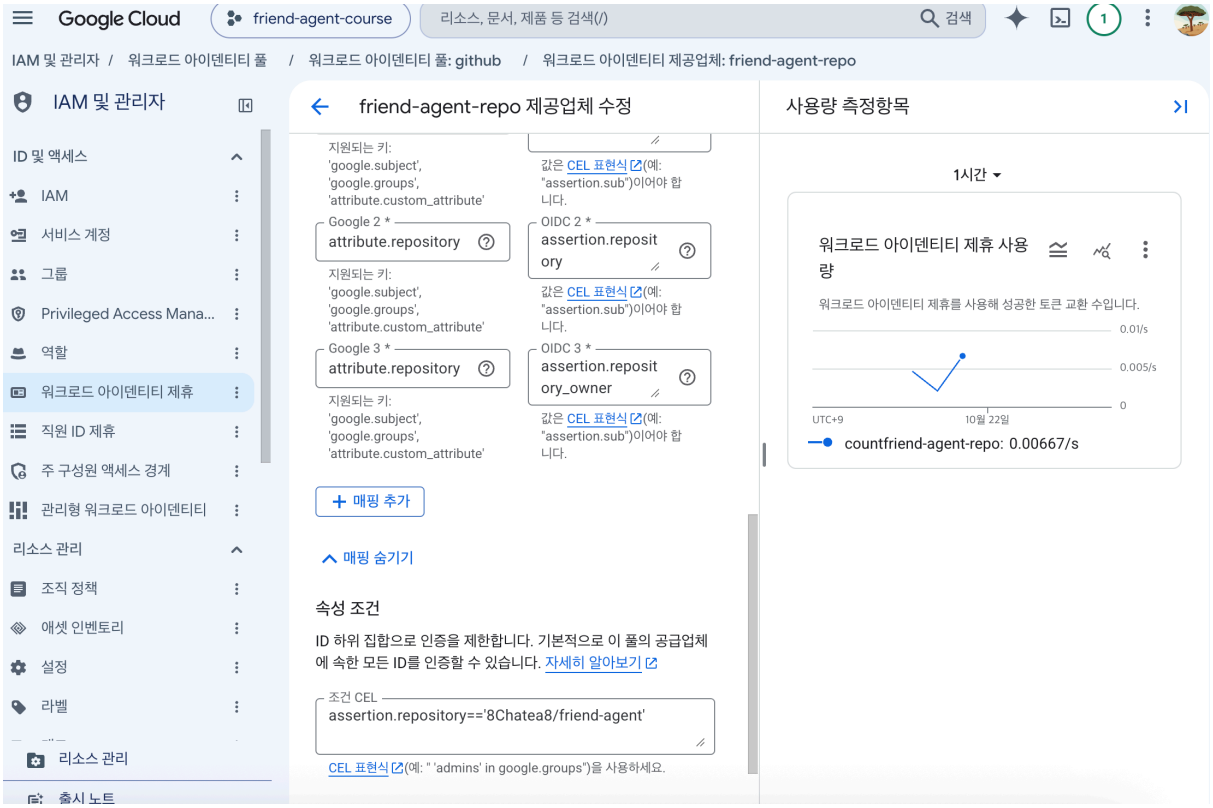


그림: IAM 및 관리자 > 워크로드 아이덴티티 제휴 > 풀 GitHub Actions > 공급자 friend-agent 저장소의 수정 화면: 속성 조건에 assertion.repository=='<내 저장소>' 가 적힌 모습

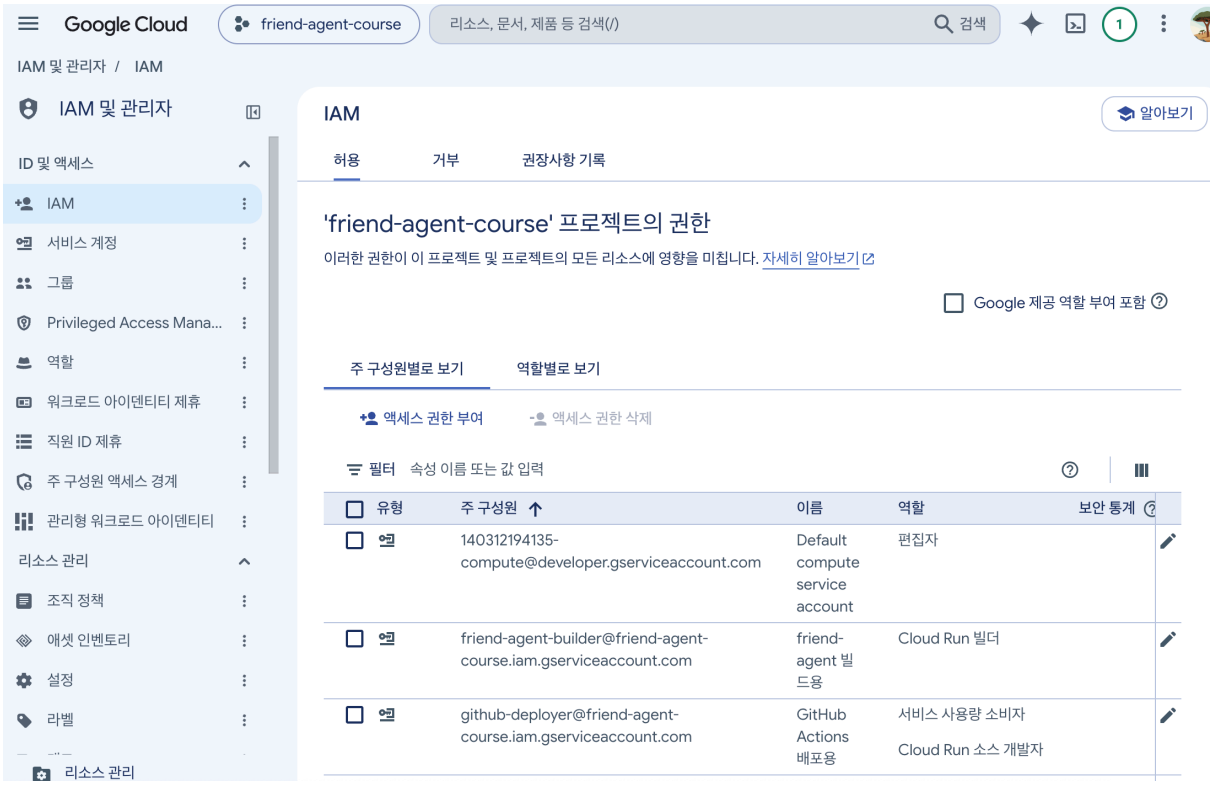


그림: IAM 및 관리자 > IAM, 주 구성원 github-deployer: Cloud Run 소스 개발자, 서비스 사용량 소비자 두 역할만 붙은 모습

풀과 공급자, 권한 연결 같은 IAM 변경은 실제로 적용되기까지 최대 5분쯤 걸리고, 이 지연을 IAM 변경 전파(propagation) 지연이라고 합니다. 바로 다음 섹션에서 인증 단계가 권한 오류로 실패하면 몇 분 기다렸다가 다시 실행해 보고, 그래도 실패하면 오류 메시지에 나온 계정 이름과 저장소 이름이 이 섹션에서 넣은 값과 같은지부터 비교합니다.

객관식 퀴즈

Q. 공급자 규칙을 내 저장소 하나가 아니라 내 GitHub 계정(예: jisu) 전체로 걸었다면 어떤 일이 생길까요?

- 1. 아무 차이가 없다. 어차피 내 계정의 저장소다
- 2. jisu 계정의 모든 저장소에서 도는 워크플로가 배포 서비스 계정을 가장할 수 있게 된다
- 3. 공급자가 만들어지지 않는다
- 4. GitHub의 모든 저장소가 배포할 수 있게 된다**제출**

- •

  1. 아무 차이가 없다. 어차피 내 계정의 저장소다

- •

  2. jisu 계정의 모든 저장소에서 도는 워크플로가 배포 서비스 계정을 가장할 수 있게 된다

- •

  3. 공급자가 만들어지지 않는다

- •

  4. GitHub의 모든 저장소가 배포할 수 있게 된다

주관식 퀴즈

Q. [이해 점검] 섹션 4에서 서비스 계정을 `github-deployer` 와 `friend-agent-runtime` 두 개로 나눴습니다. 하나로 합쳐 배포도 하고 실행도 하게 했다면 무엇이 달라질지, 그림의 두 줄을 근거로 적어 보세요.

Chapter05

워크플로 파일 넣기

권한이 준비되었으니 섹션 2에서 읽은 워크플로를 저장소에 넣습니다. 템플릿의 `starter/` 폴더에 있는 파일은 GitHub Actions가 실행하지 않고, `.github/workflows/` 폴더에 있는 파일만 실행합니다. 그래서 `starter/workflows/deploy.yml` 을 `.github/workflows/deploy.yml` 로 복사하는 변경을 섹션 3과 같은 흐름으로 PR에 올립니다. 이 PR을 머지하는 순간 main 에 커밋이 생기므로, 워크플로를 넣는 머지가 곧 첫 자동 배포가 됩니다.

직접 실행할 명령: 워크플로를 넣는 PR 열기

```
git switch main
git pull
git switch -c add-deploy-workflow

mkdir -p .github/workflows
cp starter/workflows/deploy.yml .github/workflows/deploy.yml
```

```
git add .github/workflows/deploy.yml
git commit -m "main 머지 시 Cloud Run 자동 배포"
git push -u origin add-deploy-workflow
gh pr create --base main --title "main 머지 시 Cloud Run 자동 배포" --body "starter/workflows/deploy.yml 을 그대로 복사한다"
```

코딩 에이전트에게 맡길 때: 워크플로를 넣는 PR 열기

```
add-deploy-workflow 브랜치를 만들고, starter/workflows/deploy.yml 을 .github/workflows/deploy.yml 로 복사해서 커밋해 줘.
파일 내용은 바꾸지 마. 특히 액션 버전(@v7, @v3)과 서비스 이름, 리전은 그대로 둬.
푸시하고 main 으로 가는 PR을 만들어 줘. 머지는 내가 확인한 뒤에 할게.
```

에이전트가 파일을 "내 저장소에 맞게" 고치겠다고 하면 무엇을 바꾸는지 먼저 물어봅니다. 이 파일은 저장소마다 다른 값을 모두 섹션 4에서 넣은 변수(`vars.GCP_PROJECT_ID` 등)로 읽기 때문에 고칠 곳이 없습니다.

머지 전에 PR에서 볼 것

PR의 바뀐 파일 탭에서 워크플로 한 장을 위에서 아래로 읽습니다. 머지하면 바로 실행되는 파일이라, 아래 다섯 가지가 템플릿과 같은지 확인한 뒤 머지합니다.

볼 곳	확인할 것
`on`	main 에 대한 push 만 적혀 있는가
`permissions`	`contents: read` 와 `id-token: write` 두 줄뿐인가
`uses`	`actions/checkout@v7` , `google-github-actions/auth@v3` , `google-github-actions/deploy-cloudrun@v3` 인가
`service` , `region`	`friend-agent` , `us-central1` 인가
`flags`	런타임 서비스 계정(`RUNTIME_SA`)과 빌드 서비스 계정(`BUILD_SA`)을 지정하는가

`env_vars` 의 `COMMIT_SHA=${{ github.sha }}` 는 이번 배포를 만든 커밋 SHA(커밋마다 붙는 고유한 해시값. 예: a1b2c3d)를 서비스 환경변수로 넣는 줄입니다. 에이전트 서비스의 `/version` 은 이 값을 그대로 돌려주므로, 지금 돌고 있는 버전이 어느 커밋인지 브라우저에서 바로 알 수 있습니다.

직접 실행할 명령: 머지하고 실행 지켜보기

```
gh pr merge --merge --delete-branch

# 방금 시작된 워크플로 실행을 단계별로 따라 본다
gh run watch
```

코딩 에이전트에게 맡길 때: 머지하고 실행 지켜보기

```
add-deploy-workflow PR을 머지하고, 시작된 GitHub Actions 실행이 끝날 때까지 단계별 진행 상황을 보여 줘.
실패하면 실패한 단계의 로그를 근거로 원인을 설명해 줘. 아직 고치지는 마.
```

워크플로 실행 로그에서 Cloud Run까지 이어서 확인하기

GitHub 저장소의 Actions 탭을 열면 방금 머지로 시작된 실행이 맨 위에 있습니다. 실행을 누르면 `deploy` 작업 안의 단계가 차례로 보이고, 단계 이름을 누르면 그 단계의 로그가 펼쳐집니다. 배포 단계에는 몇 분이 걸리는데, 이 동안 Google Cloud에서는 로컬 환경에서 배포할 때와 같은 빌드, 이미지 저장, 리비전 만들기가 일어납니다.

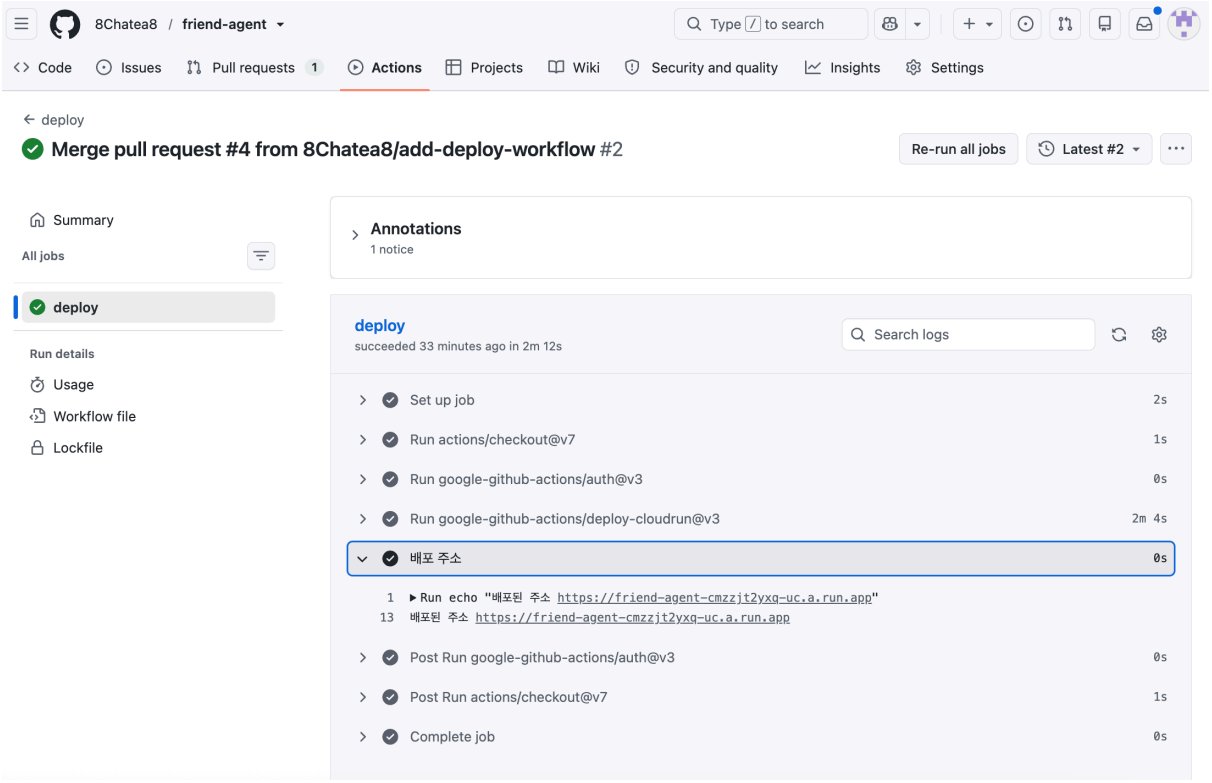


그림: GitHub 저장소 > Actions > 방금 실행 > deploy 작업: checkout, auth, deploy, 배포 주소 단계가 모두 초록 체크로 끝나고 마지막 단계에 배포 주소가 찍힌 모습

실행이 실패했다면 실패한 단계의 로그 맨 아래를 봅니다. 인증 단계에서 `Permission` 이나 `unauthorized` 가 보이면 섹션 4에서 본 IAM 변경 전파 지연이 아직 끝나지 않았거나 저장소 이름이 다르게 들어간 경우가 많고, 몇 분 뒤 실행 화면 오른쪽 위의 Re-run jobs 버튼으로 다시 돌릴 수 있습니다. 배포 단계에서 `Your account does not have permission to check details of build service account` 경고 뒤에 실패했다면 배포 서비스 계정이 빌드 서비스 계정을 쓸 권한(섹션 4의 마지막 명령)이 빠졌거나, 저장소 변수 `BUILD_SA` 가 없는 경우입니다. 이때 기본 서비스 계정을 쓸 권한을 주는 것으로 해결하지 않습니다. 기본 서비스 계정에는 Editor가 붙어 있을 수 있기 때문입니다.

성공했다면 같은 배포를 Google Cloud 쪽에서 확인합니다. Cloud Run > `friend-agent` > 리비전 탭에서 새 리비전을 누르면, 오른쪽 상세의 보안(Security) 탭에서 서비스 계정이 기본 계정이 아니라 `friend-agent-runtime` 으로 바뀌어 있고, 컨테이너(Containers) 탭의 변수 및 보안 비밀(Variables & Secrets) 항목에 `COMMIT_SHA` 가 시크릿 `llm-api-key` 와 함께 들어 있습니다. 브라우저로 서비스 URL 끝에 `/version` 을 붙여 열면 `commit` 값이 보이는데, 이 값을 GitHub 저장소 첫 화면의 최신 커밋 SHA와 비교합니다.

직접 확인할 곳: 배포 결과 이어서 확인하기

1. Cloud Run > `friend-agent` > 리비전 탭에서 새 리비전이 트래픽(traffic, 서비스로 들어오는 요청의 흐름)을 100% 받는지 봅니다.

2. 새 리비전의 보안 항목에서 서비스 계정이 `friend-agent-runtime` 인지 봅니다.
3. Cloud Build > 기록에서 가장 최근 빌드를 열고, 빌드를 실행한 서비스 계정이 `friend-agent-builder` 인지 봅니다.
4. 브라우저에서 `<서비스 URL>/version` 을 열고 `commit` 값을 GitHub 저장소의 최신 커밋 SHA와 비교합니다.
5. 화면을 열어 캐릭터와 한두 마디 나눠, 런타임 서비스 계정으로 바뀐 뒤에도 LLM API 키를 읽어 답하는지 봅니다.

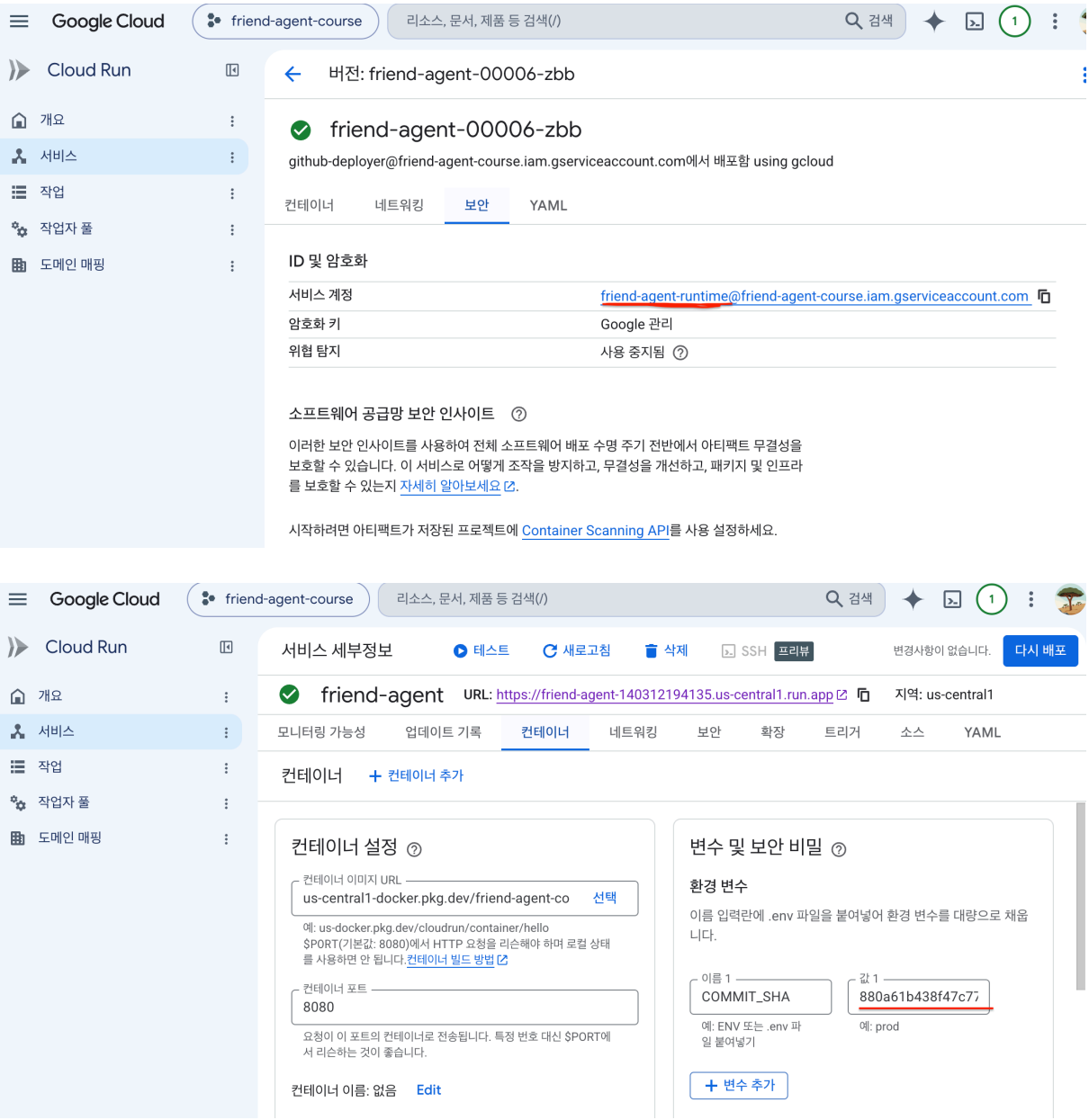


그림: Cloud Run > friend-agent > 리비전 탭 > 새 리비전: 보안 탭의 서비스 계정 friend-agent-runtime, 그리고 컨테이너 탭 > 변수 및 보안 비밀의 COMMIT_SHA

기본 서비스 계정에 남은 키 권한 회수하기

새 리비전이 런타임 서비스 계정으로 답까지 하는 것을 확인했다면, 처음 배포할 때 기본 서비스 계정에 주었던 `llm-api-key` 읽기 권한은 더 이상 필요 없습니다. 남겨 두면 기본 계정으로 실행되는 다른 서비스가 여전히 키를 읽을 수 있으므로 이 권한을 회수합니다. 이미 준 권한을 다시 지우는 것을 권한 회수(revoke)라고 하고, 권한을 지우는 명령이라 대상 시크릿과 계정 이름을 확인하고 실행합니다.

직접 실행할 명령: 기본 서비스 계정의 키 읽기 권한 회수하기

```
# 섹션 4에서 정한 PROJECT_ID, PROJECT_NUMBER 변수를 그대로 쓴다
gcloud secrets remove-iam-policy-binding llm-api-key --project $PROJECT_ID --member "serviceAccount:$PRO
JECT_NUMBER-compute@developer.gserviceaccount.com" --role roles/secretmanager.secretAccessor

# 남은 권한 확인: friend-agent-runtime 하나만 보여야 한다
gcloud secrets get-iam-policy llm-api-key --project $PROJECT_ID
```

코딩 에이전트에게 맡길 때: 기본 서비스 계정의 키 읽기 권한 회수하기

```
시크릿 llm-api-key 에서 기본 서비스 계정(<프로젝트 번호>-compute@developer.gserviceaccount.com)의 읽기 권한만 지
워 줘.
friend-agent-runtime 의 권한은 건드리지 마. 실행 전에 지우는 대상 계정과 역할을 다시 보여 주고, 끝나면 남은 권한 목록
을 보여 줘.
```

권한을 회수한 뒤 화면에서 다시 말을 걸어 답이 오면, 에이전트 서비스는 이제 자기 전용 계정으로만 키를 읽고 있다는 뜻입니다.

객관식 퀴즈

Q. 자동 배포가 끝난 뒤 `/version` 의 `commit` 값이 GitHub 저장소의 최신 커밋 SHA와 다릅니다. 가장 먼저 확인할 곳은 어디일까요?

• 1. Secret Manager의 `llm-api-key` 버전
• 2. GitHub Actions 탭에서 최신 커밋으로 시작된 실행이 성공했는지, Cloud Run 리비전 탭에서 그 리비전이 트래픽을 받는지
• 3. 로컬 환경의 `.env` 파일
• 4. 에이전트 서비스의 캐릭터 설정

- •

  1. Secret Manager의 `llm-api-key` 버전

- •

  2. GitHub Actions 탭에서 최신 커밋으로 시작된 실행이 성공했는지, Cloud Run 리비전 탭에서 그 리비전이 트래픽을 받는지

- •

  3. 로컬 환경의 `.env` 파일

- •

  4. 에이전트 서비스의 캐릭터 설정

Chapter06

머지만 하면 나간다는 것

이제 main 에 들어간 변경은 사람이 배포 명령을 치지 않아도 사용자에게 나갑니다. 편해진 만큼 잘못된 변경도 같은 속도로 나갑니다. 템플릿의 `starter/incidents/` 에는 실제로 자주 일어나는 두 종류의 잘못된 변경이 패치 파일로 들어 있습니다. 하나는 앱이 아예 시작하지 않는 변경이고, 다른 하나는 앱은 멀쩡히 시작하지만 동작이 틀린 변경입니다. 두 변경을 차례로 머지해 보고, 각각 사용자에게 어떻게 보이는지와 어떻게 되돌리는지 확인합니다.

시작하지 않는 변경 머지하기

첫 번째 패치(`rename-key.patch`)는 LLM API 키를 읽는 환경변수 이름을 `LLM_API_KEY` 에서 `OPENAI_API_KEY` 로 바꿉니다. 코드만 보면 그럴듯한 정리이고 로컬 환경의 `.env` 이름만 같이 바꾸면 로컬 환경에서는 잘 돕니다. 그런데 Cloud Run에는 `LLM_API_KEY` 라는 이름으로 시크릿이 연결되어 있으므로, 새 버전은 키를 찾지 못하고 시작하자마자 종료합니다.

직접 실행할 명령: 시작하지 않는 변경을 PR로 올려 머지하기

```
git switch main
git pull
git switch -c rename-key
git apply starter/incidents/rename-key.patch
git commit -am "LLM API 키 환경변수 이름을 OPENAI_API_KEY 로 변경"
git push -u origin rename-key
gh pr create --base main --title "LLM API 키 환경변수 이름 변경" --body "starter/incidents/rename-key.patc
h"
gh pr merge --merge --delete-branch
gh run watch
```

코딩 에이전트에게 맡길 때: 시작하지 않는 변경을 PR로 올려 머지하기

```
rename-key 브랜치를 만들어 starter/incidents/rename-key.patch 를 적용하고 커밋해 줘. 패치 외에는 아무것도 바꾸지
마.
푸시하고 main 으로 가는 PR을 만들어 머지한 뒤, 시작된 배포 실행을 끝까지 보여 줘.
실패하면 Cloud Run 로그를 근거로 원인을 설명해 줘. 고치지는 마.
```

Actions 실행은 배포 단계에서 빨간색으로 끝납니다. 이때 사용자는 무엇을 보고 있을까요. 화면을 다시 열어 말을 걸어 보면 에이전트 서비스는 아무 일 없다는 듯 답합니다. Cloud Run은 새 리비전이 스타트업 프로브(startup probe, 새 인스턴스가 요청을 받을 준비가 됐는지 Cloud Run이 확인하는 검사)를 통과해야 트래픽을 전환하므로, 통과하지 못한 새 리비전은 트래픽 0%로 남고 이전 리비전이 계속 요청을 처리하기 때문입니다. 원인은 Cloud Run 로그 탭의 오류 줄에서 찾을 수 있습니다.

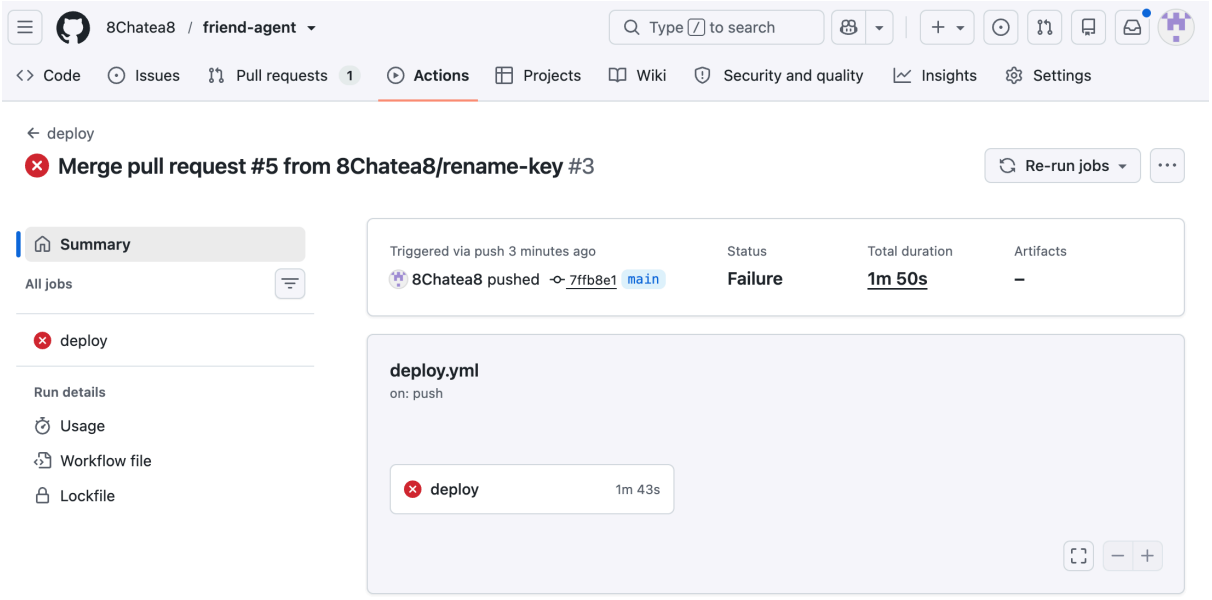


그림: GitHub 저장소 > Actions 에서 deploy 단계가 빨간색으로 실패한 모습

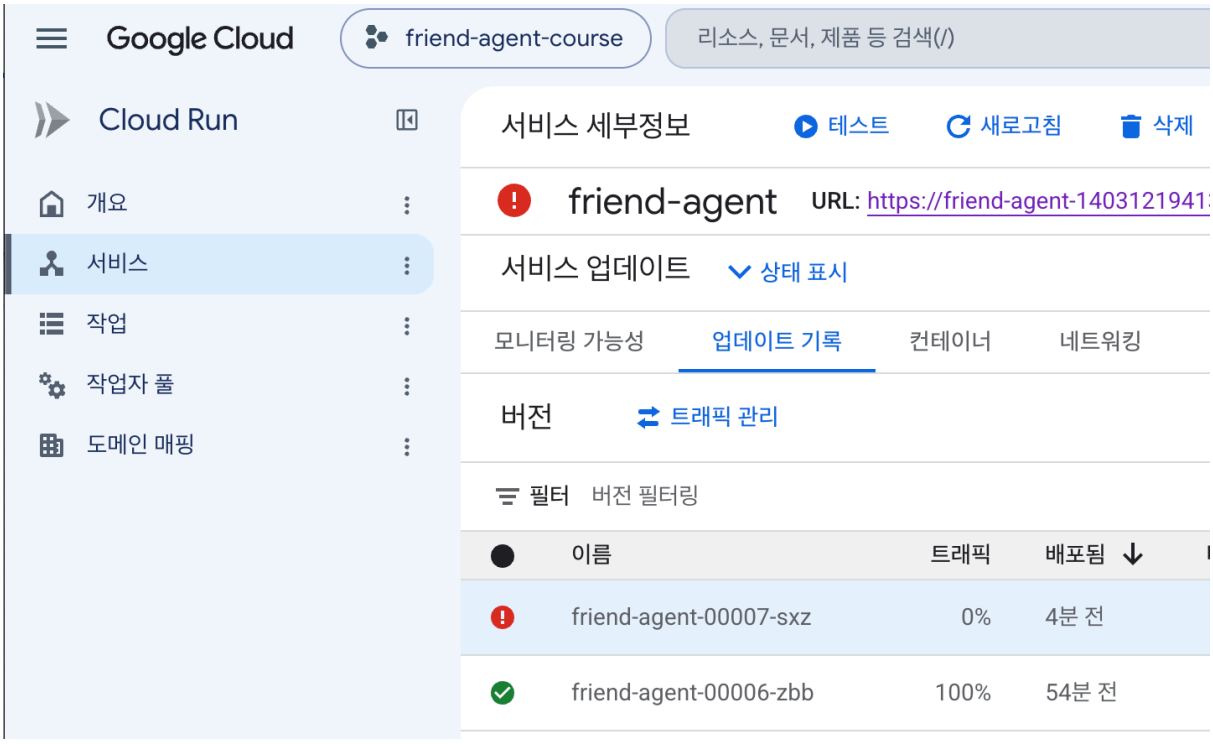


그림: Cloud Run > friend-agent > 리비전 탭에서 새 리비전은 오류 표시와 함께 0%, 이전 리비전은 100%인 모습

서비스는 멀쩡해 보여도 main 에는 시작하지 않는 코드가 들어 있습니다. 이 상태로 다른 변경을 머지하면 그 배포도 똑같이 실패하므로, main 을 되돌려야 합니다. 머지된 PR 화면 아래쪽의 Revert 버튼을 누르면, 그 PR이 바꾼 줄을 반대로 되돌리는 새 브랜치와 새 PR이 만들어집니다. 새 PR을 머지하면 main 이 원래 코드로 돌아가고, 머지로 다시 배포가 실행됩니다.

직접 확인할 곳: Revert 버튼으로 되돌리기

1. GitHub 저장소 > Pull requests > Closed 탭에서 "LLM API 키 환경변수 이름 변경" PR을 엽니다.
2. PR 화면 아래쪽의 Revert 버튼을 누릅니다.
3. 만들어진 Revert PR의 바뀐 파일 탭에서 `OPENAI_API_KEY` 가 `LLM_API_KEY` 로 돌아가는지 확인합니다.
4. Revert PR을 머지하고 Actions 탭에서 새 배포가 초록색으로 끝나는지 봅니다.

코딩 에이전트에게 맡길 때: Revert 로 되돌리기

```
"LLM API 키 환경변수 이름 변경" PR을 되돌리는 PR을 만들어 줘. 그 PR이 바꾼 줄만 반대로 되돌리고 다른 것은 바꾸지 마.
PR을 만든 뒤 바뀐 줄을 보여 주고, 내가 확인하면 머지해 줘.
```

에이전트에게 맡길 때는 `git revert` 로 revert 커밋을 만들어 PR로 올리는 방법을 쓸 수 있고, 결과는 Revert 버튼과 같습니다. main 에 직접 푸시해서 되돌리겠다는 제안은 받지 않습니다. 되돌린 기록도 PR로 남겨야 나중에 무엇이 언제 나갔다 돌아왔는지 찾을 수 있습니다.

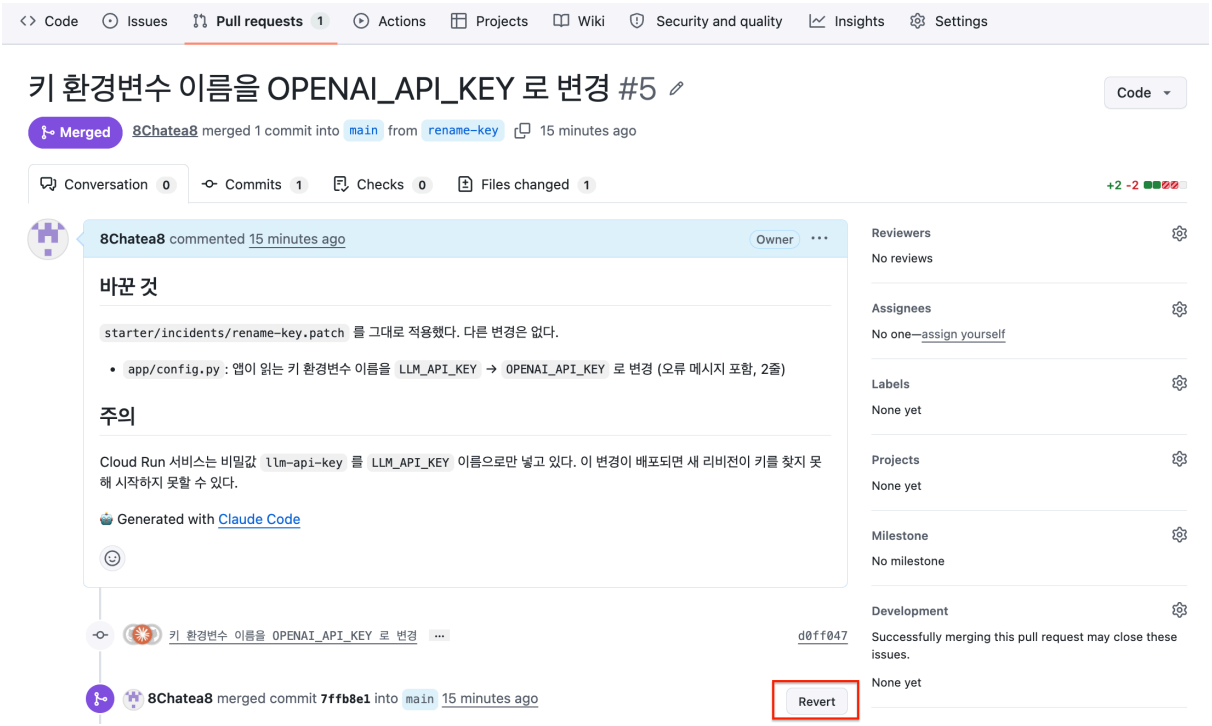


그림: GitHub 저장소 > 머지된 "LLM API 키 환경변수 이름 변경" PR 화면 아래쪽: Revert 버튼

시작은 되지만 틀린 변경 머지하기

두 번째 패치(`reminder-timezone.patch`)는 약속을 저장하기 전에 시각에서 시간대 정보를 빼는 한 줄입니다. "저장 형식을 단순하게"라는 주석까지 달려 있어 리뷰에서도 그냥 지나가기 쉽습니다. 이번에도 같은 흐름으로 머지하고, 배포가 끝나면 화면에서 약속을 하나 걸어 봅니다.

직접 실행할 명령: 틀린 변경을 PR로 올려 머지하기

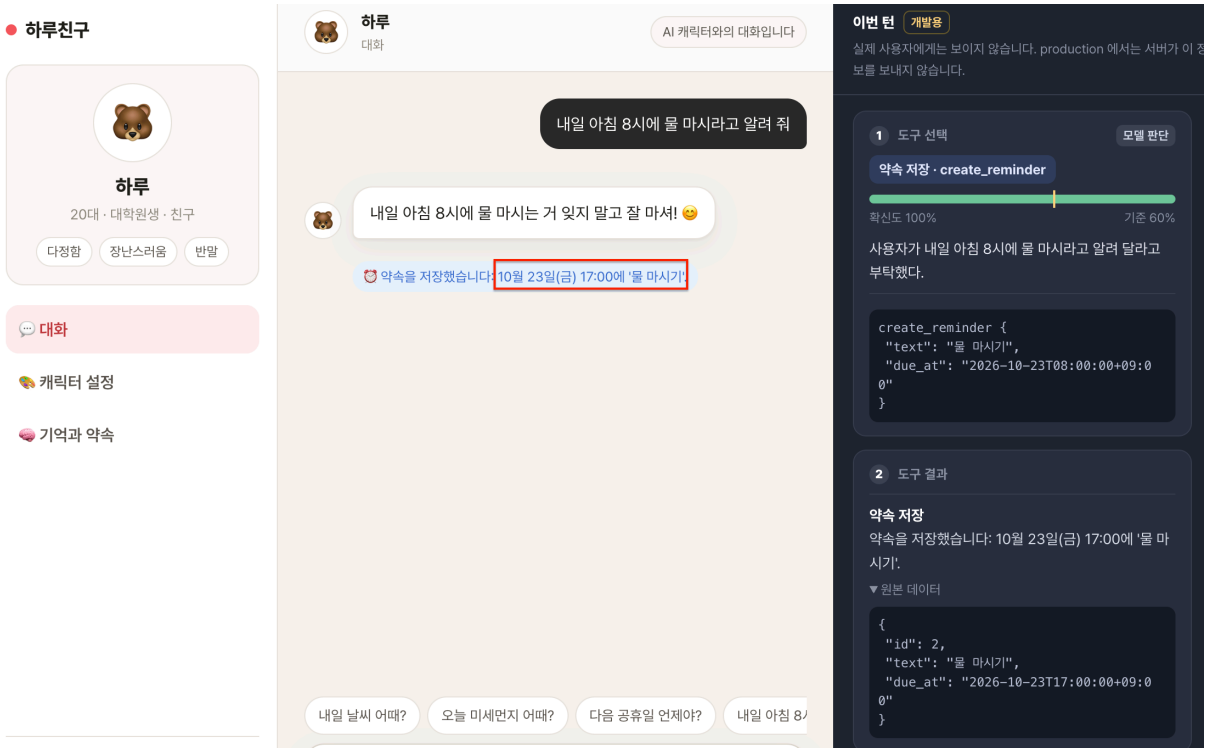
```
git switch main
git pull
git switch -c reminder-timezone
git apply starter/incidents/reminder-timezone.patch
```

```
git commit -am "약속 시각 저장 형식 단순화"
git push -u origin reminder-timezone
gh pr create --base main --title "약속 시각 저장 형식 단순화" --body "starter/incidents/reminder-timezone.pa
tch"
gh pr merge --merge --delete-branch
gh run watch
```

코딩 에이전트에게 맡길 때: 틀린 변경을 PR로 올려 머지하기

```
reminder-timezone 브랜치를 만들어 starter/incidents/reminder-timezone.patch 를 적용하고 커밋해 줘. 패치 외에는
아무것도 바꾸지 마.
푸시하고 main 으로 가는 PR을 만들어 머지한 뒤, 시작된 배포 실행을 끝까지 보여 줘.
```

이번에는 Actions 실행이 초록색으로 끝나고 새 리비전이 트래픽을 받습니다. 서비스 URL로 연 화면에서 "내일 아침 8시에 물 마시라고 알려 줘"라고 보내 보세요. 캐릭터의 답은 아침 8시라고 말하는데, 답 아래 약속 안내와 기억과 약속 화면에는 다음 날 17:00이 적혀 있습니다. 이대로라면 알림은 9시간 늦게 갑니다.



- 그림: "내일 아침 8시"로 부탁한 약속이 17:00으로 저장된 모습

같은 코드를 로컬 환경에서 실행해 같은 말을 보내면 08:00으로 제대로 저장됩니다. 시간대를 뺀 시각은 그 코드가 도는 컴퓨터의 시간대로 읽히는데, 로컬 환경은 한국 시간이고 Cloud Run 서버는 세계 표준시(UTC)를 쓰기 때문입니다. 한국 시간 08:00이 서버에서는 UTC 08:00으로 저장되고, 화면에 한국 시간으로 바꿔 보여 줄 때 9시간이 더해져 17:00이 됩니다. 앱은 정상적으로 시작했으니 배포 단계에서는 아무것도 걸리지 않았고, 틀린 약속 시각은 그대로 사용자에게 나갔습니다.

이 변경도 섹션 6 앞부분과 같은 방법으로 Revert PR을 만들어 되돌리고, 배포가 끝나면 같은 말을 다시 보내 08:00으로 저장되는지 확인합니다. 지금은 약속이 인스턴스 메모리에만 있어서 새 리비전이 뜰 때 17:00 약속도 함께 사라집니다. 데이터를 데이터베이스에 두는 서비스였다면 이미 잘못 저장된 약속은 코드를 되돌려도 그대로 남아, 따로 찾아서 고쳐야 했을 것입니다.

오늘 겪은 변경 정리

오늘 겪은 두 변경을 아래 표로 정리합니다. 두 변경이 어디에서 드러났고 어떻게 되돌렸는지 나란히 놓고 보면, 자동 배포가 무엇을 빠르게 해 주고 무엇은 막아 주지 못하는지가 보입니다.

변경	Actions 결과	사용자에게 보인 것	원인을 찾은 곳	되돌린 방법과 걸린 시간
LLM API 키 환경변수 이름 변경				
약속 시각 저장 형식 단순화				

주관식 퀴즈

Q. [이해 점검] 두 변경 모두 PR 화면에서 바뀐 줄을 보고 머지했습니다. 첫 번째 변경은 사용자에게 나가지 않았고 두 번째 변경은 나갔습니다. 두 변경이 이렇게 갈린 이유를 설명하고, 두 번째 변경을 머지 전에 알아챌 수 있었다면 누가 또는 무엇이 어떤 확인을 했어야 했을지 적어 보세요.

0/300**다음**

📚 자동 배포 참고 자료

- GitHub Actions 이해: 워크플로, 이벤트, 작업, 단계, 러너, 액션이 어떻게 맞물리는지 공식 설명을 한국어로 볼 수 있습니다.
- GitHub Actions 빠른 시작: 워크플로 파일을 저장소에 넣고 Actions 탭에서 실행 기록을 보는 기본 흐름을 한국어로 볼 수 있습니다.
- GitHub Actions 워크플로 구문: `on`, `permissions`, `runs-on`, `steps` 에 쓸 수 있는 값을 모두 찾아볼 수 있습니다.
- OpenID Connect를 사용한 보안 강화: 키 없이 OIDC 토큰으로 클라우드에 인증하는 방식과 `id-token: write` 의 의미를 설명합니다.
- google-github-actions/auth: 오늘 쓴 인증 액션의 설정 방법과 Workload Identity Federation 설정 명령이 정리되어 있습니다.
- google-github-actions/deploy-cloudrun: 배포 액션의 입력값(`env_vars`, `flags`, `no_traffic` 등)과 필요한 권한을 확인할 수 있습니다.
- 끌어오기 요청 되돌리기: Revert 버튼이 무엇을 만들고, 버튼이 보이지 않을 때는 어떻게 하는지 설명합니다.

Chapter07

과제 고르기

오늘 만든 자동 배포를 조금 더 써 보고, 섹션 4에서 만든 권한 설정이 실제로 무엇을 막고 무엇을 남기는지 확인하는 과제입니다. 수업 시간 안에는 과제 1을 하고, 나머지는 수업이 끝난 뒤에 골라서 해 봅니다. 과제마다 필요한 안내가 그 과제 아래에 모두 있습니다.

과제	단계	하는 일
1. 기능 하나 넣고 자동 배포로 적용하기	기본	작은 기능을 만들어 머지하고, 사람이 배포 명령을 치지 않아도 사용자 화면에 나오는지 확인한다
2. 다른 저장소에서 배포 시도하기	중간	내 저장소가 아닌 곳에서 같은 워크플로를 돌려 Google Cloud가 거절하는지 본다
3. 누가 배포했는지 감사 로그로 추적하기	중간	배포 기록에서 배포 서비스 계정과 그 뒤의 저장소 신원을 찾는다
4. 다른 CD 방법 조사하기	조사	GitHub Actions 말고 어떤 방법으로 자동 배포를 만들 수 있는지 비교한다

과제 1. 기능 하나 넣고 자동 배포로 적용하기

오늘 만든 자동 배포가 실제 기능 추가에서도 그대로 동작하는지 확인합니다. 화면에서 바로 보이는 작은 기능 하나를 정합니다. 예를 들면 화면 아래쪽에 지금 배포된 커밋 SHA 앞 7자리를 보여 주는 표시(`/version` 응답을 읽어 표시), 입력창 위 추천 문장 몇 개 추가, 캐릭터 만들기 화면에 새 말투 선택지 하나 추가 같은 것입니다. 무엇을 바꿀지와 화면 어디에 보일지를 먼저 정하고 코딩 에이전트에게 맡깁니다.

만든 기능은 섹션 3과 같은 흐름으로 작업 브랜치에서 커밋하고 PR을 올려 main 에 머지합니다. main 에 바로 푸시해도 배포는 시작되지만, 무엇이 언제 바뀌었는지 PR로 남기기 위해 머지로 넣습니다. 머지한 뒤에는 배포 명령을 직접 실행하지 않고 아래 네 곳만 확인합니다.

직접 확인할 곳: 과제 1 확인

1. GitHub 저장소의 Actions 탭에서 머지 커밋으로 시작된 deploy 실행이 성공했는지 봅니다.
2. Cloud Run > `friend-agent` > 리비전 탭에서 새 리비전이 트래픽 100%를 받는지 봅니다.
3. 서비스 URL 끝에 `/version` 을 붙여 열고, `commit` 이 머지 커밋 SHA와 같은지 봅니다.
4. 서비스 URL을 새로고침해 추가한 기능이 화면에 보이는지 봅니다. 브라우저가 이전 화면을 기억하고 있으면 강력 새로고침(macOS는 Cmd+Shift+R, Windows는 Ctrl+F5)을 합니다.

과제 2. 다른 저장소에서 배포 시도하기

섹션 4에서 공급자 규칙에 내 저장소 이름을 넣었습니다. 정말 다른 저장소의 워크플로는 막히는지 직접 확인합니다. Google Cloud 설정은 하나도 바꾸지 않고, GitHub에 연습용 저장소를 하나 더 만들어 내 저장소와 같은 배포 워크플로와 저장소 변수를 넣은 뒤 main 에 커밋해 워크플로를 돌립니다.

예상대로라면 인증 단계(`google-github-actions/auth`)에서 실패하고, 로그에 공급자의 속성 조건 때문에 거절되었다는 내용이 남습니다. 배포 단계까지 가지 않았는지와 Cloud Run 리비전 목록이 그대로인지도 확인합니다. 시퀀스 다이어그램의 검사 ①에서 멈춘 것입니다. 확인이 끝나면 연습용 저장소를 지웁니다. 저장소를 지우기 전에는 지우는 저장소 이름이 연습용인지 한 번 더 확인합니다.

과제 3. 누가 배포했는지 감사 로그로 추적하기

Google Cloud는 서비스를 만들거나 바꾸는 작업을 감사 로그(audit log)로 자동으로 남깁니다. 오늘 머지로 일어난 배포가 감사 로그에 누구의 작업으로 남았는지 찾아봅니다. 로그 탐색기(Logs Explorer)에서 Cloud Run 관리 활동 기록을 거르고, 가장 최근 배포 시각의 기록을 엽니다. 기록 안에서 작업한 주체가 내 Google 계정이 아니라 `github-deployer` 인지, 그리고 그 계정을 가장한 신원에 내 저장소 이름과 브랜치가 적혀 있는지 봅니다.

직접 명령으로 배포했던 리비전의 기록과 나란히 놓으면 차이가 보입니다. 사람이 배포하면 내 계정이 남고, 자동 배포하면 배포 서비스 계정과 저장소가 남습니다. 문제가 생긴 배포를 거슬러 올라갈 때 이 기록이 어느 저

장소의 어느 커밋에서 시작되었는지 알려 줍니다.

직접 확인할 곳: 과제 3 감사 로그 찾기

1. Google Cloud 콘솔 > 로깅(Logging) > 로그 탐색기를 엽니다.
2. 쿼리 창에 `protoPayload.serviceName="run.googleapis.com"` 과 `logName:"cloudaudit.googleapis.com%2Factivity"` 두 줄을 넣고 실행합니다.
3. 가장 최근 배포 시각의 기록을 펼쳐 `authenticationInfo` 아래의 `principalEmail` 이 `github-deployer@…` 인지 봅니다.
4. 같은 기록에서 `serviceAccountDelegationInfo` 를 펼쳐 내 저장소 이름과 `refs/heads/main` 이 들어 있는지 봅니다.

과제 4. 다른 CD 방법 조사하기

오늘은 GitHub Actions가 배포를 실행하고, 권한은 WIF로 받았습니다. 자동 배포를 만드는 방법은 이것만 있지 않습니다. Google Cloud 안에서 저장소를 연결해 Cloud Build가 배포하는 방법, 배포 단계와 승인을 전문으로 관리하는 Cloud Deploy, 회사에서 많이 쓰는 GitLab CI/CD와 Jenkins, 저장소에 적힌 상태를 클러스터가 스스로 따라가는 GitOps 방식의 Argo CD, 그리고 처음에 썼던 Vercel 같은 플랫폼 연동이 있습니다.

이 가운데 세 가지 이상을 골라 공식 문서나 코딩 에이전트에게 물어 조사하고, 아래 표를 채웁니다. 채운 뒤에는 "에이전트 서비스의 배포 방법을 지금 방식에서 하나로 바꾼다면 무엇이 쉬워지고 무엇을 잃는가"를 두세 문장으로 적어 봅니다.

방법	배포를 실행하는 곳	배포 순서가 적힌 곳	Google Cloud 권한을 얻는 방법	테스트, 승인, 단계별 배포를 넣기	비용
GitHub Actions + WIF (오늘)	GitHub 러너	저장소의 워크플로 파일	OIDC 토큰으로 서비스 계정 가장	워크플로에 직접 적는다	공개 저장소 무료

📚 과제 4를 시작할 자료

- Cloud Run 지속적 배포: Cloud Run 콘솔에서 저장소를 연결해 Cloud Build가 배포하게 하는 방법입니다.
- Cloud Deploy 개요: 배포 단계와 승인, 단계별 배포를 관리하는 Google Cloud 서비스입니다.
- GitLab CI/CD: GitLab 저장소의 파이프라인 파일로 배포하는 방법입니다.
- Argo CD: 저장소에 적힌 상태를 따라가는 GitOps 방식 도구입니다.

Chapter08

오늘 배운 것

자동 배포 구조. 워크플로 파일은 트리거(`on`), 러너(`runs-on`), 단계(`steps`), 권한(`permissions`)을 정합니다. 연습용 `hello.yml` 을 PR과 실행 버튼으로 돌려 보면서, 러너가 실행마다 비어 있는 상태로 시작해 커밋된 파일

만 내려받는다는 것과 단계 하나가 실패하면 뒤 단계를 건너뛴다는 것을 로그에서 확인했습니다. 배포 워크플로에서는 브랜치에서 고친 변경이 PR과 머지를 거쳐 main 에 들어가면 GitHub이 제공하는 러너에서 배포가 실행됩니다. 결과는 워크플로 실행 로그, Cloud Run 리비전, `/version` 의 커밋 SHA로 이어서 확인했습니다.

권한을 나누는 방식. 키 파일 대신 Workload Identity Federation으로, 내 저장소에서 온 OIDC 토큰일 때만 배포 서비스 계정을 가장할 수 있게 했습니다. 배포 서비스 계정과 런타임 서비스 계정을 나눠 각각 필요한 역할만 주고, 기본 서비스 계정에 남아 있던 키 읽기 권한은 회수했습니다.

실패에서 배운 것. 시작하지 않는 변경은 배포 단계에서 멈추고 이전 리비전이 계속 요청을 처리했지만, main 에 남은 코드는 Revert PR로 되돌려야 했습니다. 시작은 되지만 틀린 변경은 로컬 환경에서도, 배포 단계에서도 걸리지 않고 사용자에게 그대로 나갔습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 Vercel이 푸시 뒤 몇 분 동안 무엇을 했는지, 어떤 권한으로 내 코드를 가져갔는지 짐작해 적었습니다. 그 답을 다시 읽고, 오늘 직접 만든 워크플로와 권한 설정에 비춰 짐작이 맞았던 부분과 새로 알게 된 부분을 적어 보세요.